EN3150 Pattern Recognition
Unsupervised Learning Techniques
k-means, PCA, ICA, Multidimensional Scaling (MDS)

Most of the contents are based on

1. C. M. Bishop, Pattern Recognition and Machine Learning, Springer NY, 2006.
2. Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.

M. T. U. Sampath K. Perera,

Department of Electronic and Telecommunication Engineering,

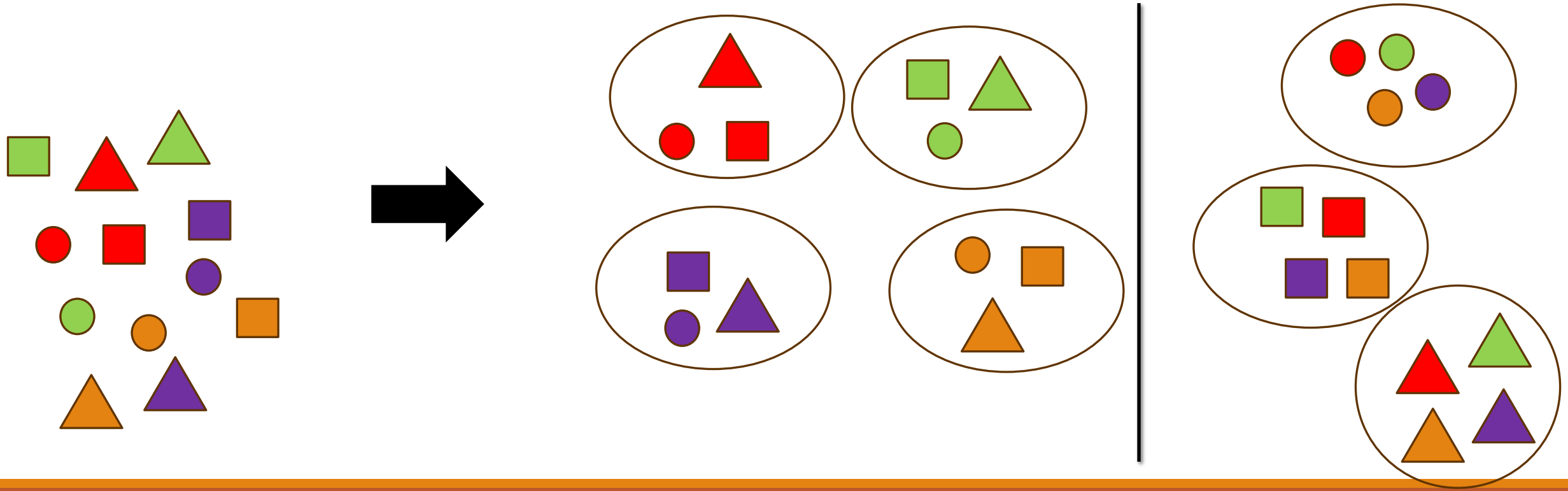
University of Moratuwa.

(sampathk@uom.lk).

Semester 5 – Batch 21.

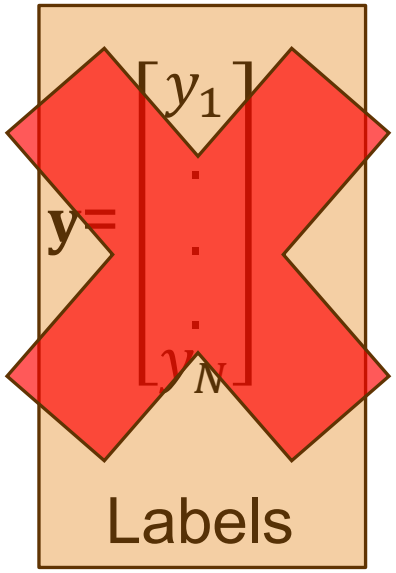
Learning from data: Unsupervised learning

- Unsupervised learning involves training an algorithm on unlabeled data without explicit output labels
  - The algorithm's objective is to find patterns, structures, or relationships within the data.



Unsupervised learning





$$\boldsymbol{X} = \begin{bmatrix} x_{1,1\ldots} & x_{d,1} \\ x_{1,2\ldots} & x_{d,2} \\ \cdot & \cdot \\ \cdot & \cdot \\ \cdot & \cdot \\ x_{1,N}\ \ldots & x_{d,N} \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \cdot \\ \cdot \\ \cdot \\ \boldsymbol{x}_N \end{bmatrix}$$

Data ($\boldsymbol{D}$)

$$\boldsymbol{D} = \{\boldsymbol{x}_n : n = 1{:}\,N\}$$

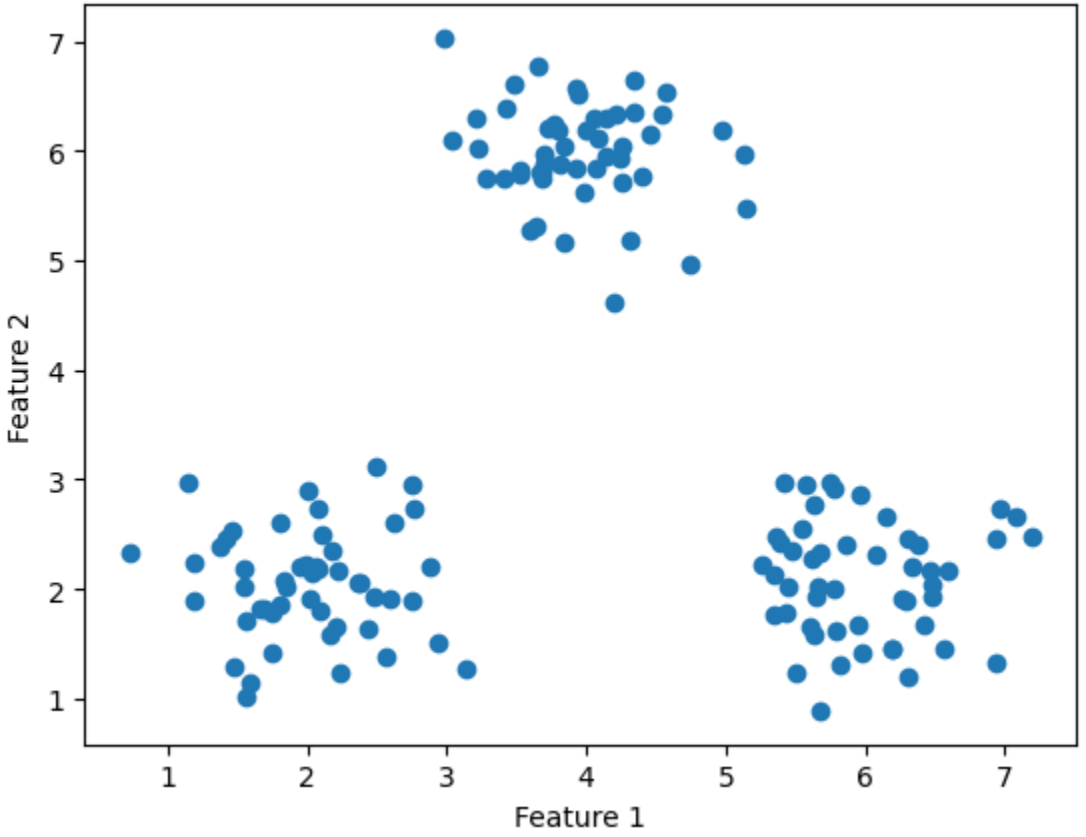
and typically, $\boldsymbol{x}_n \in \mathbb{R}^d\}$.

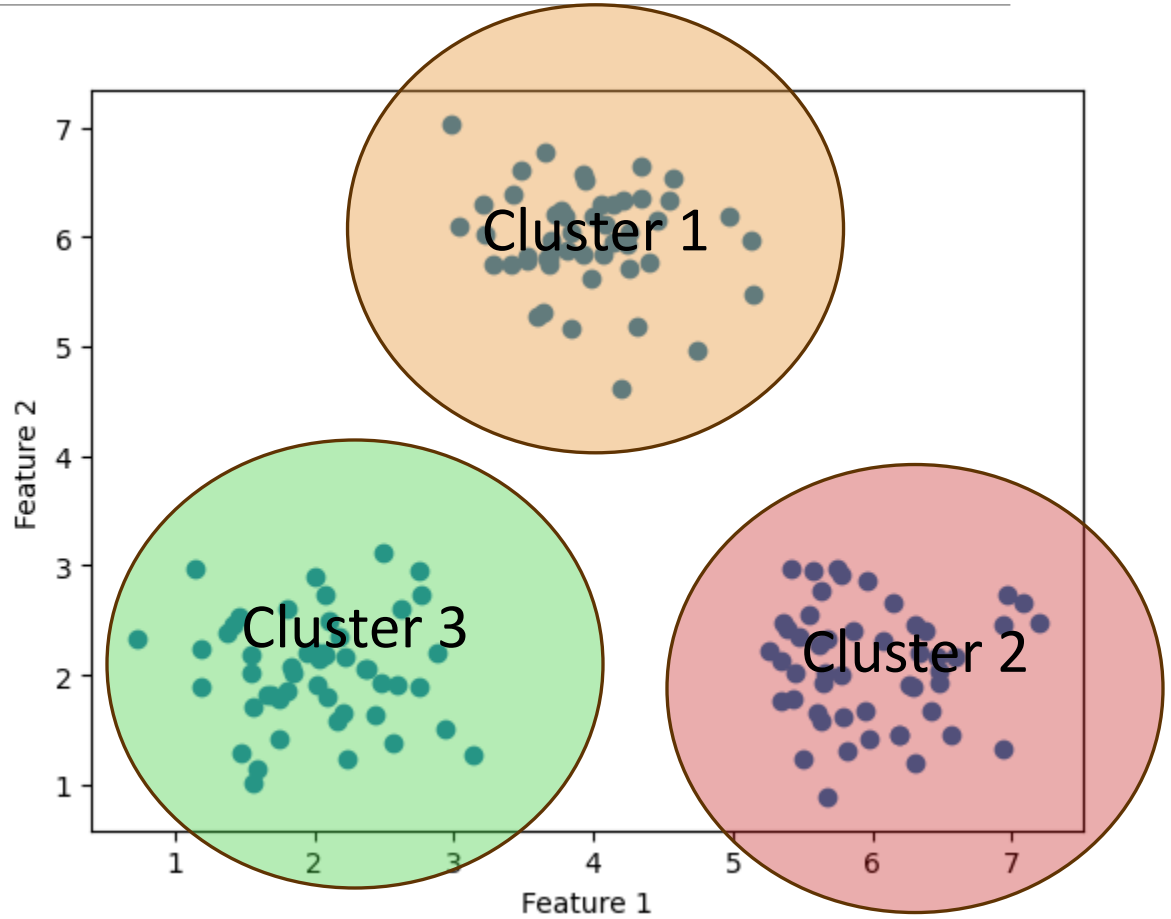
- We have Data($\boldsymbol{X}$) but no labels $\mathbf{y}$.
- Goal: find patterns, structures, or relationships within the data.

Unsupervised learning

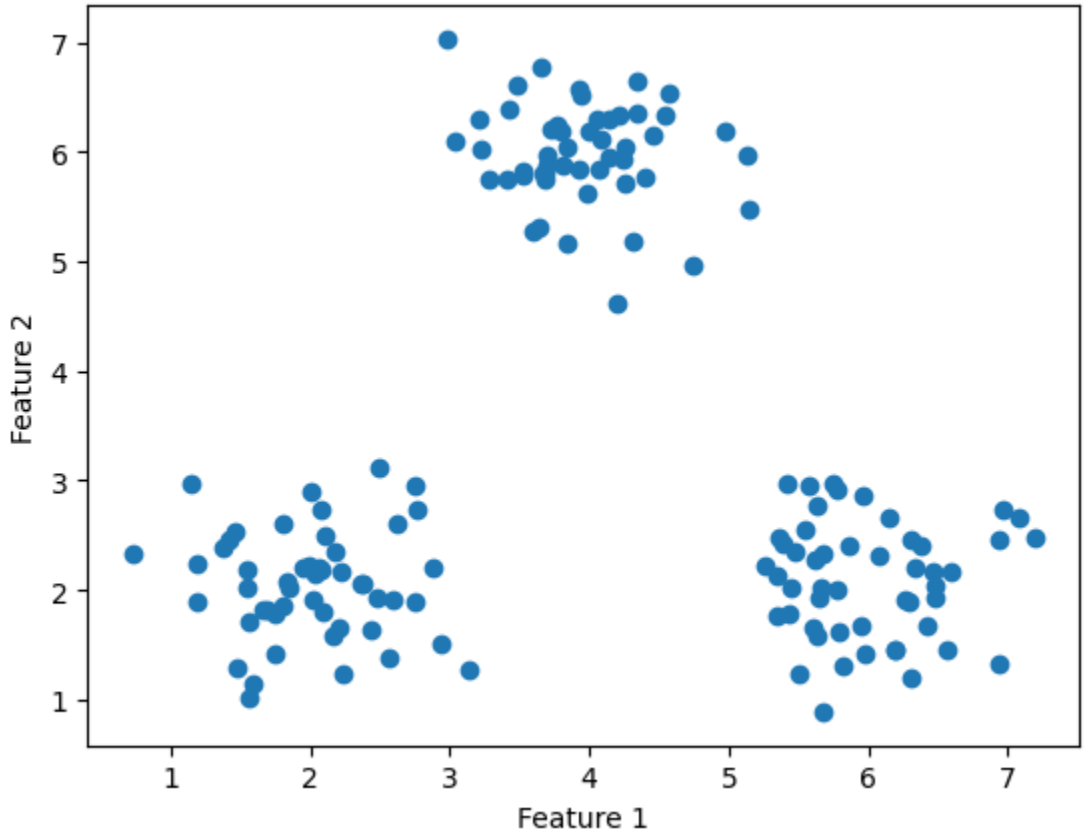
- **Clustering:** grouping data samples into clusters based on their similarities (**k-Mean**s, Hierarchical clustering).
- **Dimensionality Reduction:** reduce the number of features in a dataset while preserving its essential characteristics (Principal Components Analysis (**PCA**), Independent Component Analysis (**ICA**), Reconstruction ICA (RICA), Sparse Coding , Auto encoders).
- **Density Estimation:** model the probability distribution of data in order to understand the underlying structure (Gaussian Mixture Models (**GMMs**) and Kernel Density Estimation (**KDE**)).

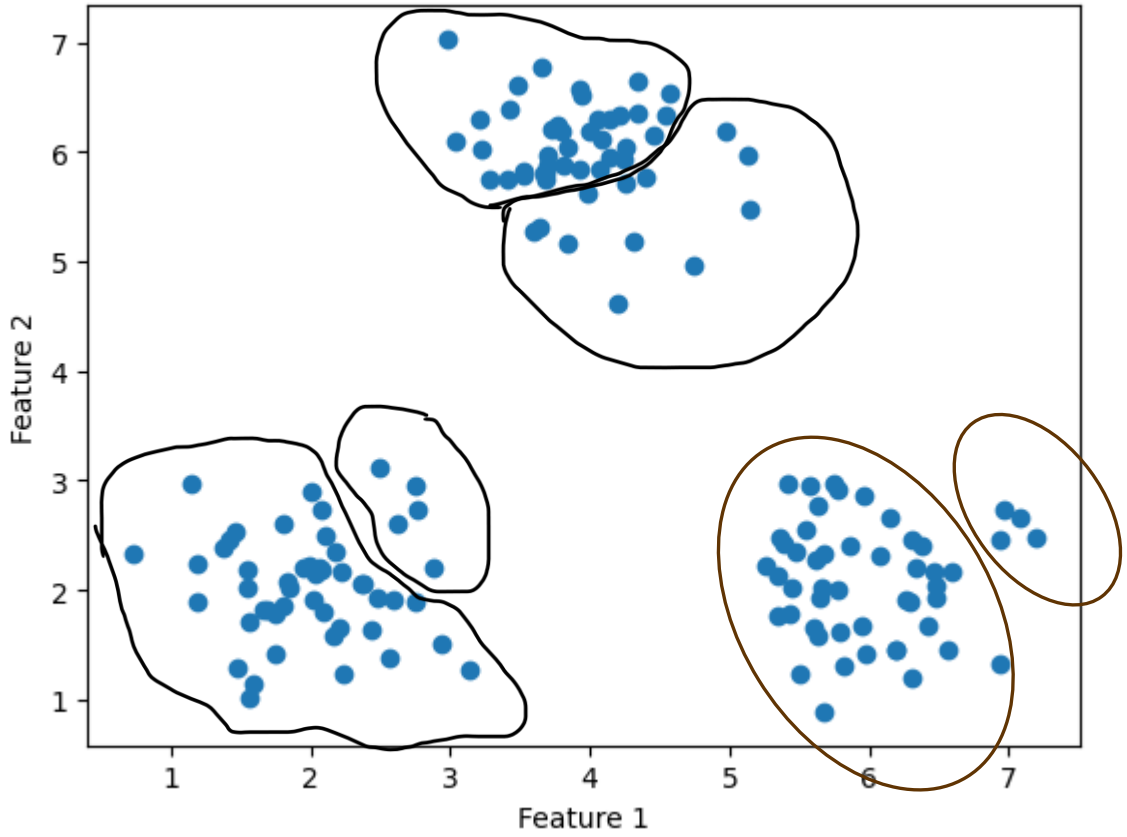
Clustering


Feature 2
Feature 1
1
2
3
4
5
6
7


Cluster 1
Cluster 3
Cluster 2
Feature 2
Feature 1
1
2
3
4
5
6
7

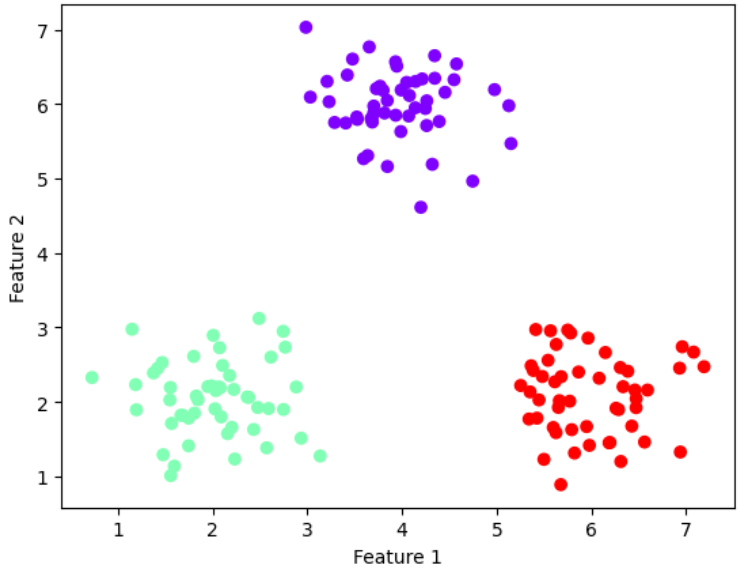
Clustering





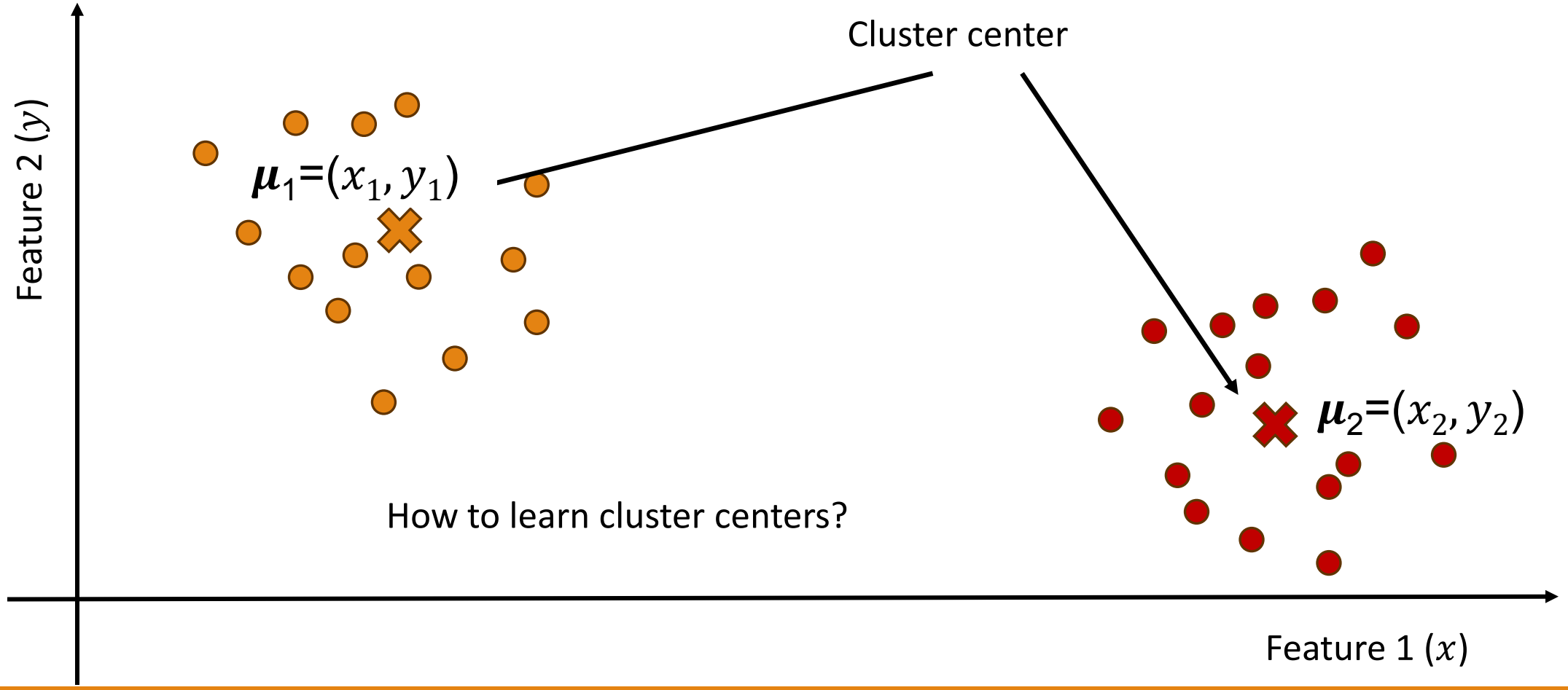
k-means Clustering

- k-means clustering: Divide dataset into k clusters (predefined value of k).
- What is the criteria:
  - Cluster: A group of data points with small inter-point distances compared to distances outside the cluster.





k-means Clustering



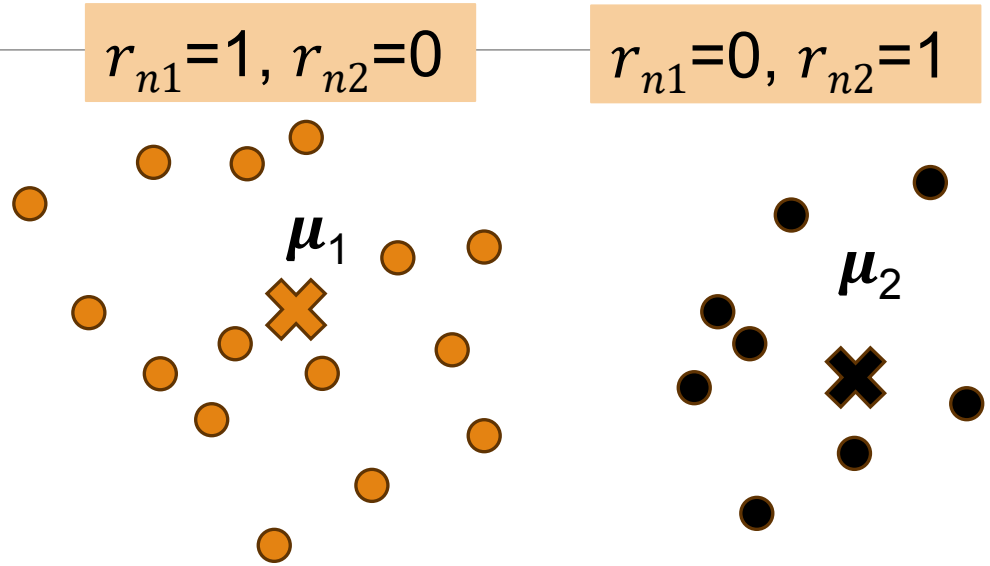
k-means Clustering

- $\boldsymbol{x}_n$ – features and $\boldsymbol{\mu}_k$ – k-th cluster center
- How to learn $\boldsymbol{\mu}_k$?

Sum of distances from each point to its assigned cluster center

distortion measure

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$$



Cluster assignment

$$r_{nj} = \begin{cases} r_{nk} = 1, & \boldsymbol{x}_n \text{ assigned to } k \text{ th cluster} \\ r_{nj} = 0, & \text{for } j \neq k \end{cases}$$

this is called 1-of-k coding scheme

k-means Clustering

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$$

since J is a linear function of r_{nk} closed form optimization is possible

2 optimizations

- How to learn r_{nk} and $\boldsymbol{\mu}_k$
- Iterative algorithm (maximization)
  1. Fix r_{nk} learn $\boldsymbol{\mu}_k$
  2. Fix $\boldsymbol{\mu}_k$ learn r_{nk} (expectation)

two steps in each iteration corresponding to two optimizations

Fix $\boldsymbol{\mu}_k$ learn r_{nk}

$$r_{nk} = \begin{cases} 1, if\ k = \arg\min_j \|\boldsymbol{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0 \end{cases}$$

in simple terms

Assigning data point to it's closest cluster center.

E (expectation) step of the EM algorithm

expectation maximization

k-means Clustering

J is a quadratic function of μ_k
⇒ optimize by setting derivative to zero

$$J = \sum_{n=1}^{N}\sum_{k=1}^{K} r_{nk}\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$$

- How to learn r_{nk} and $\boldsymbol{\mu}_k$
  - Iterative algorithm
    1. Fix r_{nk} learn $\boldsymbol{\mu}_k$
    2. Fix $\boldsymbol{\mu}_k$ learn r_{nk}

Fix r_{nk} learn $\boldsymbol{\mu}_k$

$$J = \sum_{n=1}^{N} r_{n1}\|\boldsymbol{x}_n - \boldsymbol{\mu}_1\|^2 + \ldots + \sum_{n=1}^{N} r_{nk}\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2 + \ldots + \sum_{n=1}^{N} r_{nK}\|\boldsymbol{x}_n - \boldsymbol{\mu}_K\|^2$$

$$\frac{\delta J}{\delta \boldsymbol{\mu}_k} = 0 \rightarrow 2\sum_{n=1}^{N} r_{nk}(\boldsymbol{x}_n - \boldsymbol{\mu}_k) = 0$$

$$\boldsymbol{\mu}_k = \frac{\sum_{n=1}^{N} r_{nk}\boldsymbol{x}_n}{\sum_{n=1}^{N} r_{nk}}$$

sum of points assigned to cluster k

no. of points assigned to cluster k

M (maximization) step of the EM algorithm

Mean of the data points in the cluster (k-means algorithm)

k-means Clustering

Initialize K cluster centers ($\boldsymbol{\mu}_k$)

Repeat until convergence:

For each data point $\boldsymbol{x}_n$:

Assign $\boldsymbol{x}_n$ to the nearest centroid

For each cluster k:

Update the centroid $\boldsymbol{\mu}_k$ as the mean of all data points assigned to cluster k

E (expectation) step of the EM algorithm

M (maximization) step of the EM algorithm

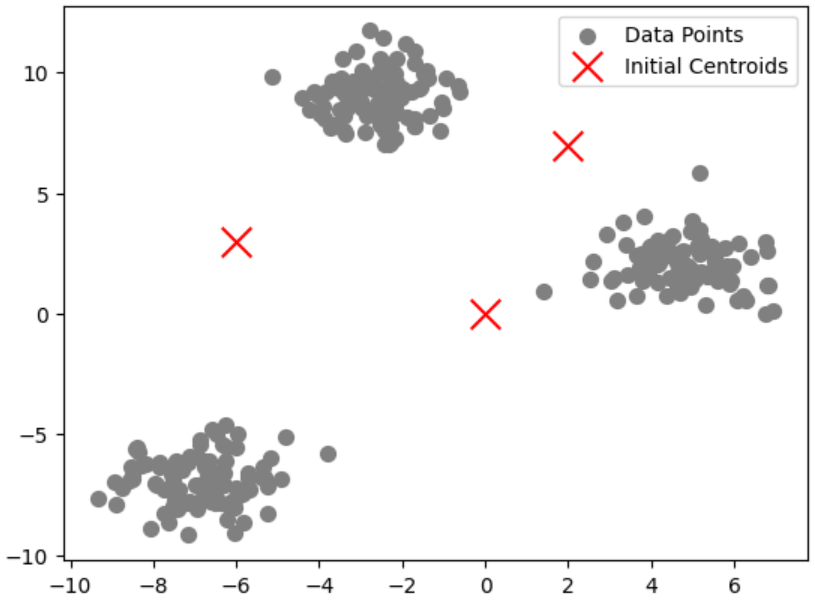
- When to stop update: no further change in the cluster centers or some fixed number of iterations are reached.
- Usually converges to local minimum*. Initialization is important.
- Implementation of k-means can be slow.

↳ run for fixed no. of iterations

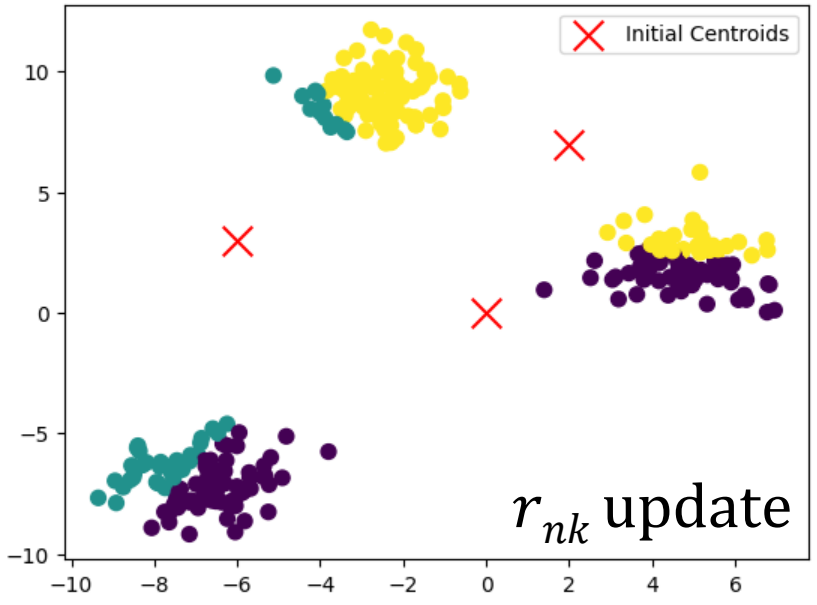
*The convergence properties of the K-means algorithm were studied by MacQueen (1967) "MacQueen, James. "Some methods for classification and analysis of multivariate observations." *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*. Vol. 1. No. 14. 1967."

k-means Clustering

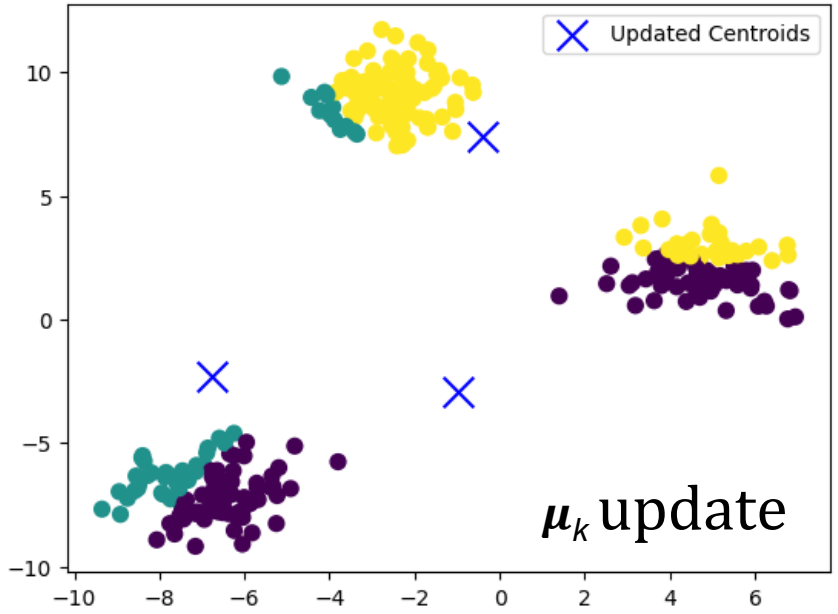
Initialization



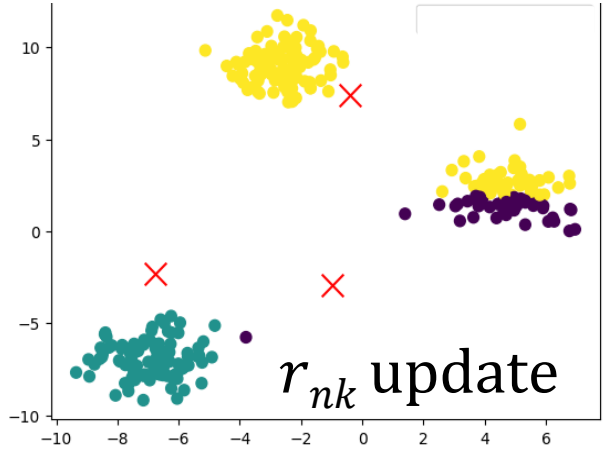
Cluster assignment (E (expectation) step)



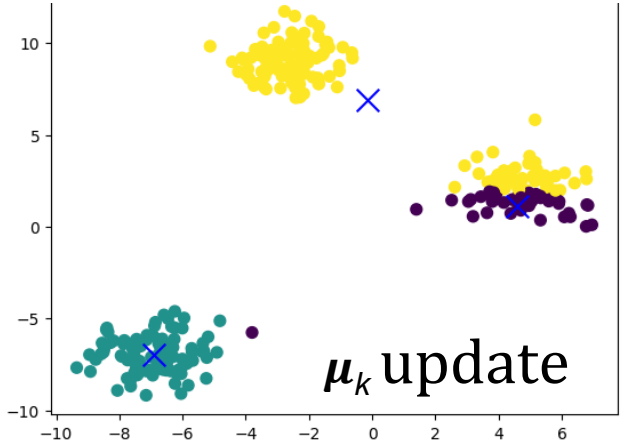
Update the centroid (M (maximization) step)



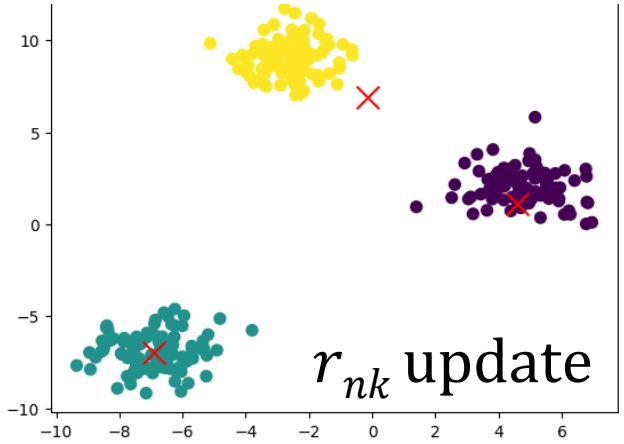
Cluster assignment (E (expectation) step)



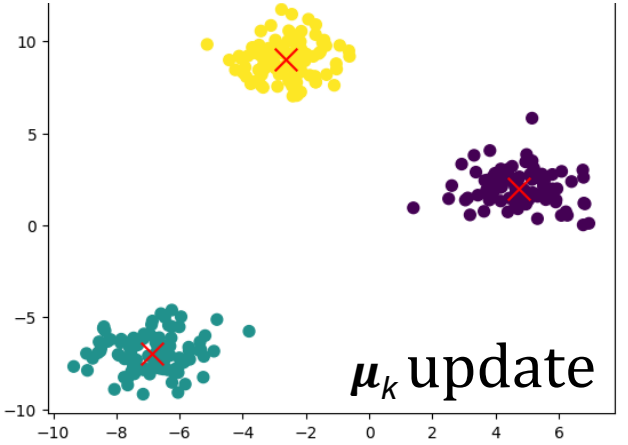
Update the centroid (M (maximization) step)

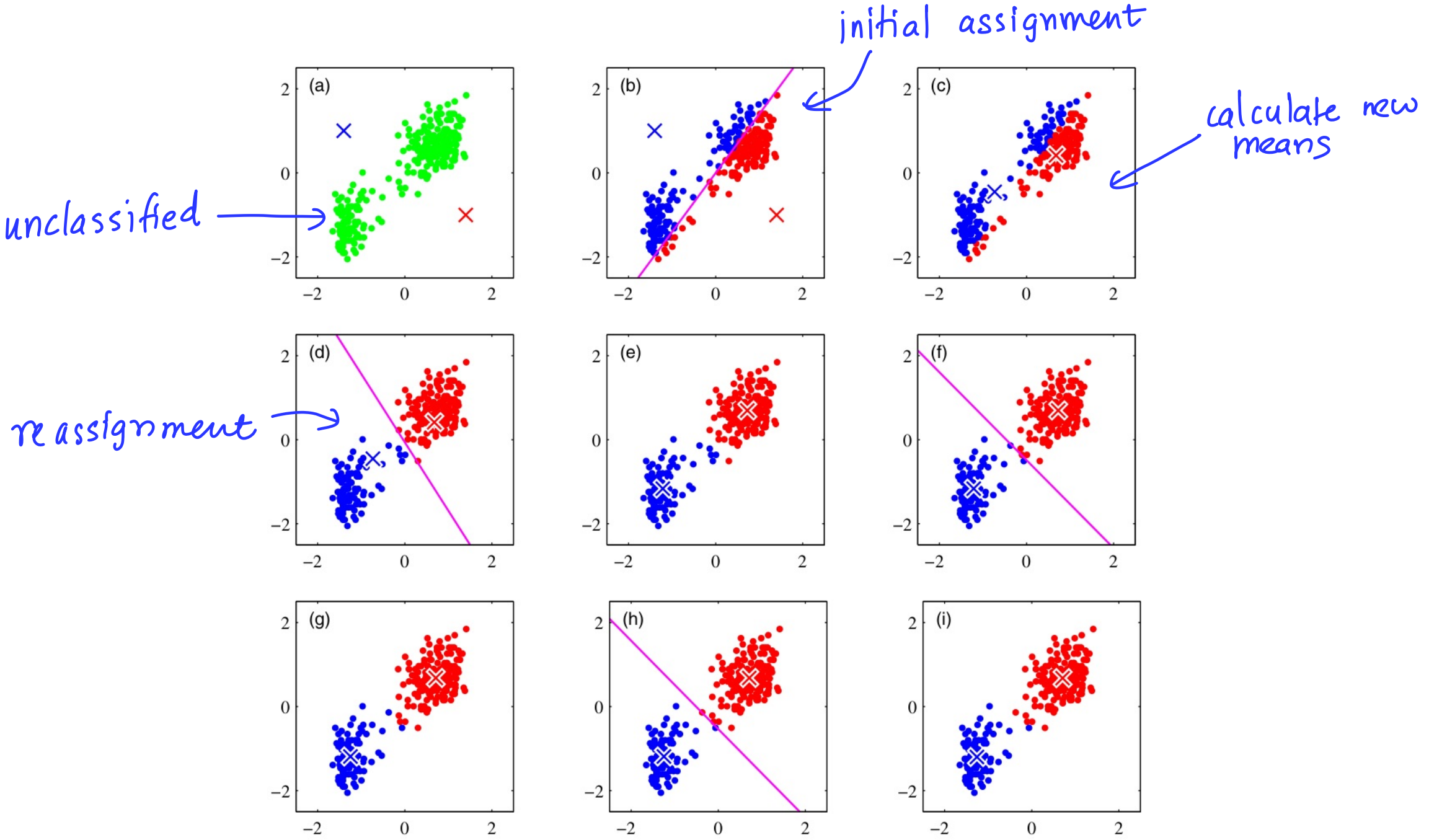


Cluster assignment (E (expectation) step)

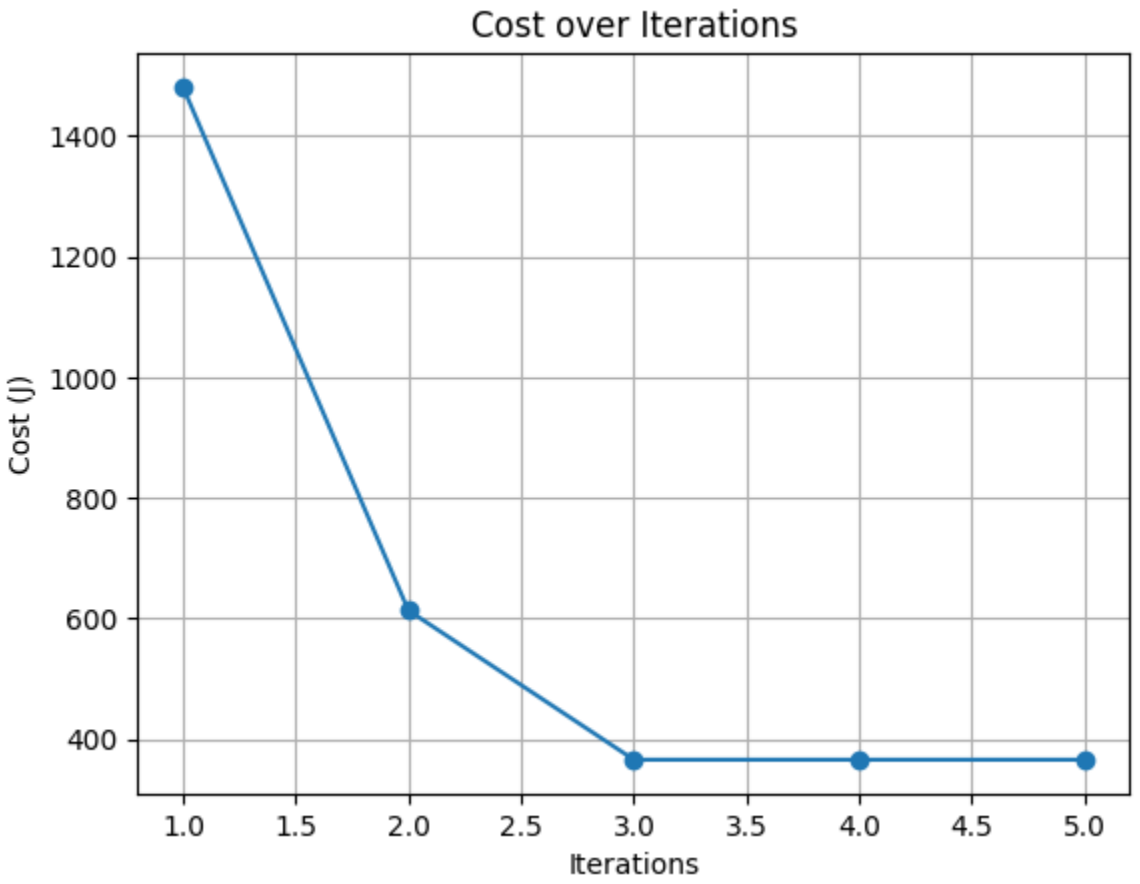


Update the centroid (M (maximization) step)




initial assignment
calculate new means
unclassified
reassignment
(a)
(b)
(c)
(d)
(e)
(f)
(g)
(h)
(i)

k-means Clustering



* can reduce iterations if initial assignment of centers is taken from the datapoints

Mini Batch k-means Clustering

have to compute for all the points

- Batch version of k-means can be very slow.
- Mini-batches are used to reduce the computation time.
- Mini-batches are subsets of the input data, randomly sampled in each training iteration.

- Sklearn: Comparison of the k-Means and MiniBatch kMeans clustering algorithms
- Clustering text documents using k-means

k-means Clustering

- Batch version of k-means can be very slow.
- Online stochastic version of k-means*

learning parameter

We update the nearest cluster center $\boldsymbol{\mu}_k$ for each data point $\boldsymbol{x}_n$.

$$\boldsymbol{\mu}_{k,new} = \boldsymbol{\mu}_{k,old} + \alpha\,(\boldsymbol{x}_n - \boldsymbol{\mu}_{k,old})$$

α learning rate parameter







sequential update

*"MacQueen, James. "Some methods for classification and analysis of multivariate observations." *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*. Vol. 1. No. 14. 1967."

α set to monotonically decrease as we consider more data points

k-means Clustering

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$$

Is this a good loss function?

disadvantages of Euclidean distance as dissimilarity measure

- It's unsuitable for **categorical variables** and **sensitive to outliers.**
- Generalization: Introduce a broader dissimilarity measure $\vartheta(\boldsymbol{x}_n, \boldsymbol{\mu}_k)$.

$$\tilde{J} = \sum_{n=1}^{N} \sum_{k=1}^{K} \vartheta(\boldsymbol{x}_n, \boldsymbol{\mu}_k)$$

k-medoids algorithm

k-means Clustering

- k-means assigns each data point to a **single cluster** in every iteration **(hard assignment)**.
- Weakness:
  - Some points are significantly closer to a cluster center than others.
  - Some points might be equidistant between cluster centers.
  - For such midway points, **hard assignment** to the nearest cluster might not be ideal.

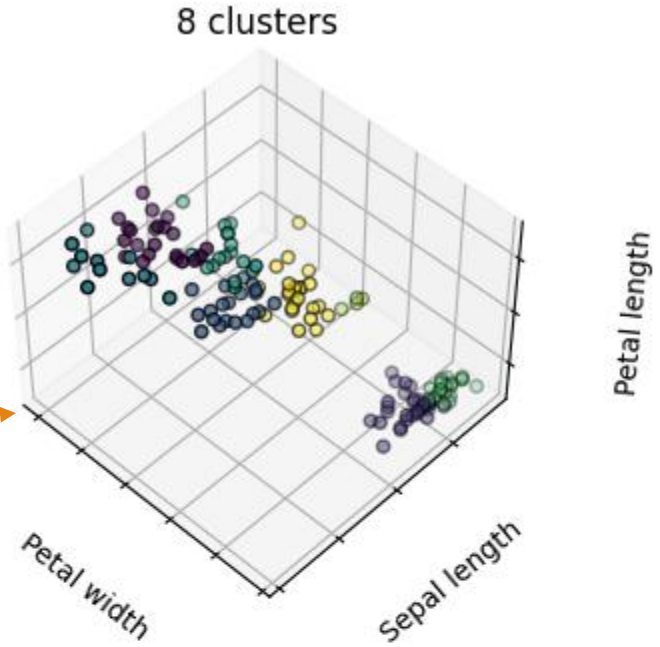
↳ use soft assignment (probabilistic)

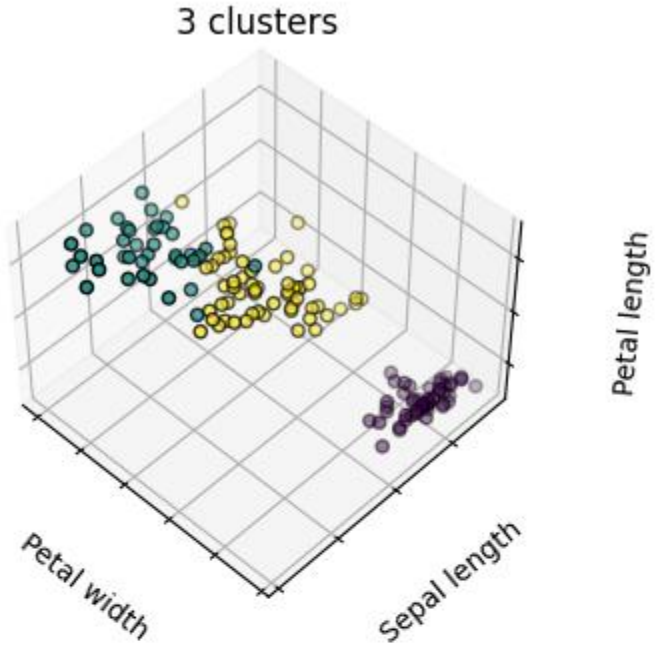
k-means Clustering

- k-means is often known as Lloyd's algorithm.
- The algorithm is typically composed of three key steps.
  - In the first step, initial centroids are selected, commonly done by choosing K samples from the dataset.
  - Following initialization, k-means iterates between the remaining two steps.
  - The second step is responsible for assigning each sample to its nearest centroid.
  - In the third step, new centroids are created by computing the mean value of all samples assigned to each previous centroid.
  - The algorithm calculates the difference between the old and new centroids and repeats the last two steps until this difference falls below a defined threshold.

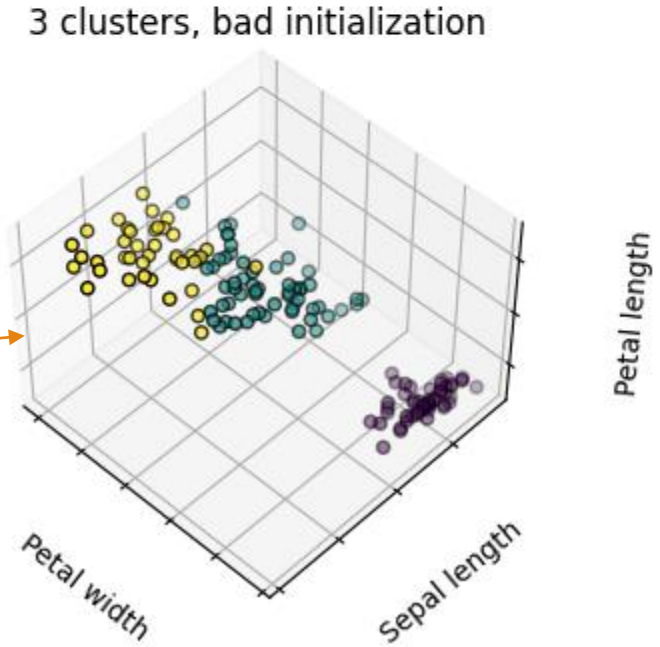
k-means Clustering

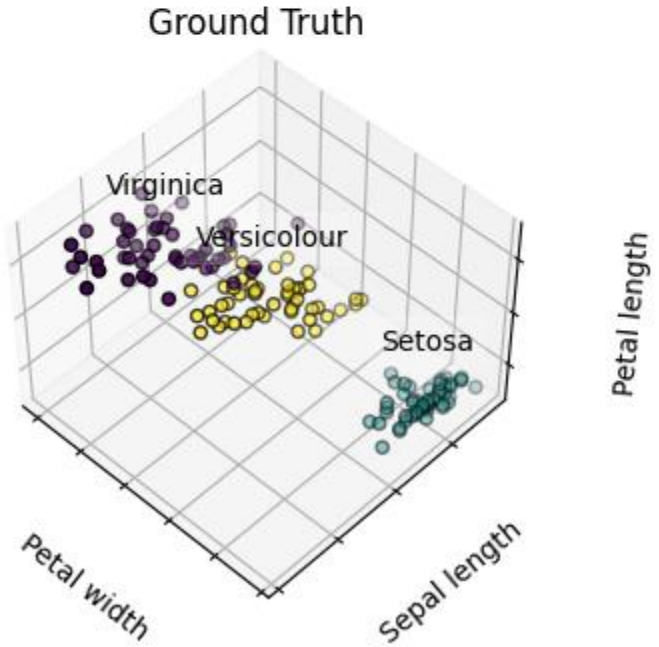
K-means algorithm would yield using 8 clusters.





The effect of a bad initialization is on the classification process





Code: K-means Clustering — scikit-learn

k-means Clustering: color quantization ns Clustering

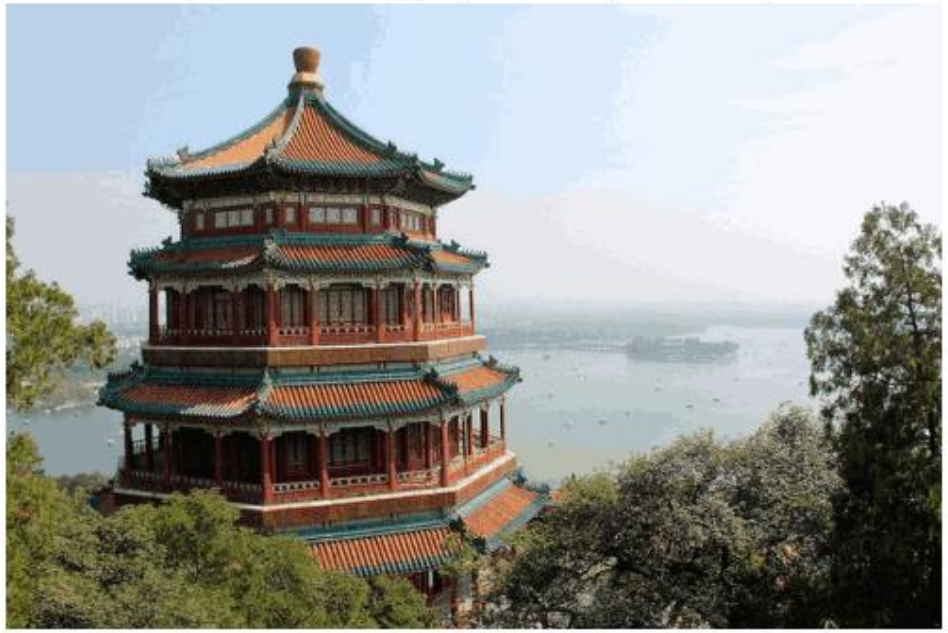
Pixel-wise Vector Quantization (VQ)

Original image (96,615 colors)

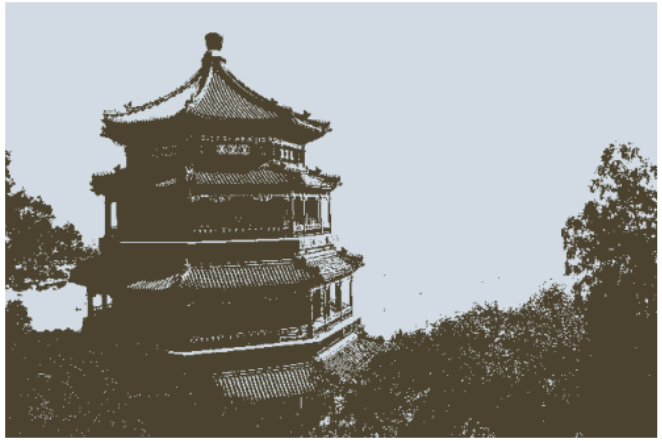


Reduce the number of colors in an image while preserving its visual appearance. (**96,615** to **64**)

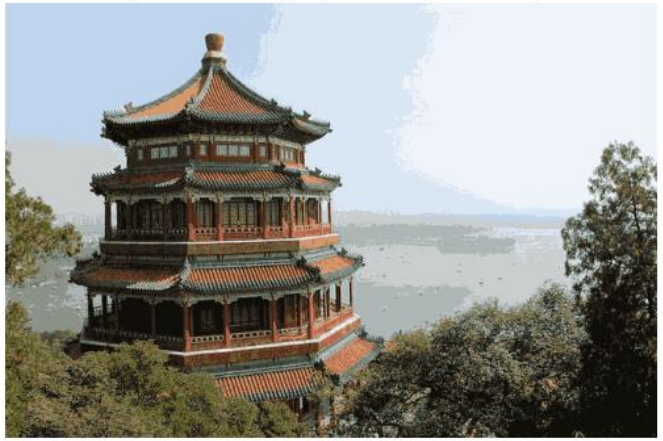
Quantized image (64 colors, K-Means)



Quantized image (2 colors, K-Means)



Quantized image (16 colors, K-Means)



Quantized image (64 colors, Random)



Code: Color Quantization using K-Means — scikit-learn 1.3.0 documentation

process is called vector quantization

μ_k's – codebook vectors

k-means Clustering

- Image contains N pixels with {R, G, B} values with 8 bits of precision.
- For transmission it requires 3 x 8 x N = 24N bits.
- An alternative approach involves running K-means on the image data.
  - Transmit the index of the nearest vector $\boldsymbol{\mu}_k$, which consumes $\log_2$ K bits per pixel.
  - Transmitting the K code book vectors $\boldsymbol{\mu}_k$ also need 3 x 8 x K = 24K bits.
  - Total number of bits needed to transmit the image is 24K + N $\log_2$ K bits.

need K clusters → $\log_2 K$ bits

- e.g., 240 × 180 image (for $K=8$, $N=240\times180$)
- Direct transmission = 240 × 180 x24 = 1036800 bits.
- k-means= 24 x 8 + 240 × 180 $\log_2$ 8 = 129792 bits.
- Compression ratio = 1036800/ 129792 = 7.98%.

Typically, $K \ll N$

μ_k's

cluster numbers for each pixel

k-means++ ← a better initialization scheme

- k-means usually converges to local minimum. Initialization is important.
- Hence, k-means performed multiple times, using various centroid initializations.
- To address this k-means++ initialization scheme can be used.
- This approach sets the centroids initially at relatively distant positions, potentially yielding improved results compared to random initialization.

Arthur, David, and Sergei Vassilvitskii. "K-means++ the advantages of careful seeding." *Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms*. 2007.

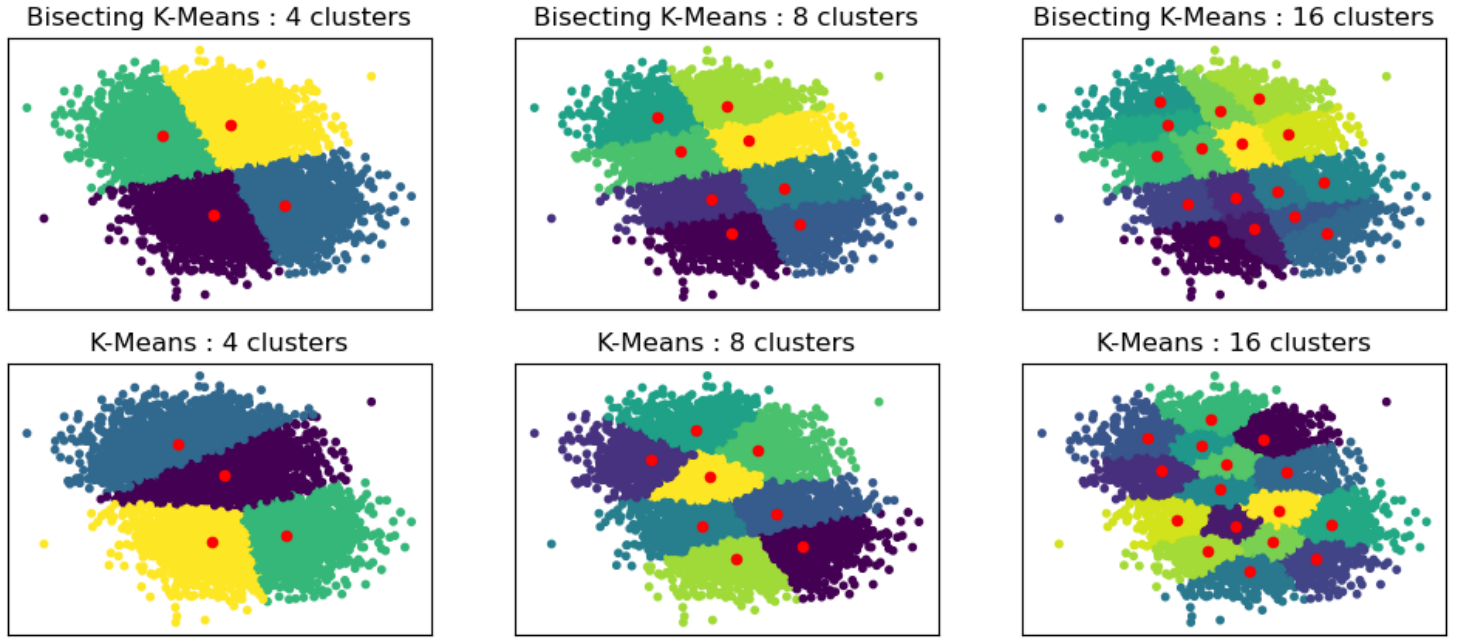
Bisecting k-Means

* for large, complex datasets
* heirachial structure of data

- k-Means clustering exhibit variations as the number of clusters increases.
- In contrast, Bisecting k-means clustering builds upon the previous clusters, leading to the creation of clusters with a more consistent large-scale structure.
- This distinction becomes visually apparent: regardless of the number of clusters, Bisecting k-means displays a dividing line that bisects the overall data cloud, a characteristic not present in regular k-means.

When dealing with large datasets or data with complex structures, Bisecting k-Means can be more efficient.

It provides insights into the hierarchical structure of the data



Code: Bisecting k-Means and Regular k-Means Performance Comparison — scikit-learn

Dimensionality Reduction

Dimensionality Reduction

- Learn mapping from high-dimensional space $\boldsymbol{x} \in \mathbb{R}^d$ to low-dimensional space $\boldsymbol{z} \in \mathbb{R}^l$, where $l \ll d$.
- Mapping can be parametric $\boldsymbol{z} = f(\boldsymbol{x};\boldsymbol{\theta})$.
- Mapping can be non-parametric too.

Also known as Karhunen-Loève transform

two definitions → orthogonal projection
→ linear projection

Principal Components Analysis (PCA)

- Principal Component Analysis (PCA) : Simplest and most widely used dimensionality reduction technique.
- Objective: Find a linear and orthogonal projection of the high dimensional data to low-dimensional space. → principal subspace
- Projection of $\boldsymbol{x} \in \mathbb{R}^d$ to low-dimensional space $\boldsymbol{z} = \boldsymbol{W}^T\boldsymbol{x} \in \mathbb{R}^l$, where $\boldsymbol{W} \in \mathbb{R}^{dxl}$.
- Unprojection $\hat{\boldsymbol{x}} = \boldsymbol{W}\boldsymbol{z}$.
- Objective to minimize the difference $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$.
- Reconstruction error (distortion)

d-sized vector

l sized vector

$$L(\boldsymbol{W}) \triangleq \frac{1}{N}\sum_{i=1}^{N}\|\boldsymbol{x}_i - \text{decode}(\text{encode}(\boldsymbol{x}_i;\boldsymbol{W});\boldsymbol{W})\|^2.$$

Objective function

Principal Components Analysis (PCA)

- Projection of $\boldsymbol{x} \in \mathbb{R}^d$ to low-dimensional space $\boldsymbol{z} = \boldsymbol{W}^T\boldsymbol{x} \in, \mathbb{R}^l$ where $\boldsymbol{W} \in \mathbb{R}^{dxl}$.

Data	x_1	x_2	…	x_i	…	x_N
Projection	z_{11}	z_{21}	…	z_{i1}	…	z_{N1}

- Suppose 1d projection, $l = 1$, $z_{i1} \in \mathbb{R}^1$, $\boldsymbol{w}_1 \in \mathbb{R}^{dx1}$

$$L(\boldsymbol{w}_1, \boldsymbol{z}_1) \triangleq \frac{1}{N}\sum_{i=1}^{N}\|\boldsymbol{x}_i - z_{i1}\boldsymbol{w}_1\|^2,$$

$$= \frac{1}{N}\sum_{i=1}^{N}(\boldsymbol{x}_i - z_{i1}\boldsymbol{w}_1)^T(\boldsymbol{x}_i - z_{i1}\boldsymbol{w}_1),$$

$$= \frac{1}{N}\sum_{i=1}^{N}(\boldsymbol{x}^T{}_i\boldsymbol{x}_i - 2\, z_{i1}\boldsymbol{w}^T{}_1\boldsymbol{x}_i + z^2{}_{i1}\boldsymbol{w}_1{}^T\boldsymbol{w}_1), \qquad \boldsymbol{w}_1{}^T\boldsymbol{w}_1 = 1.$$

(unit vector)

$$= \frac{1}{N}\sum_{i=1}^{N}(\boldsymbol{x}^T{}_i\boldsymbol{x}_i - 2\, z_{i1}\boldsymbol{w}^T{}_1\boldsymbol{x}_i + z^2{}_{i1}).$$

Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.

Principal Components Analysis (PCA)

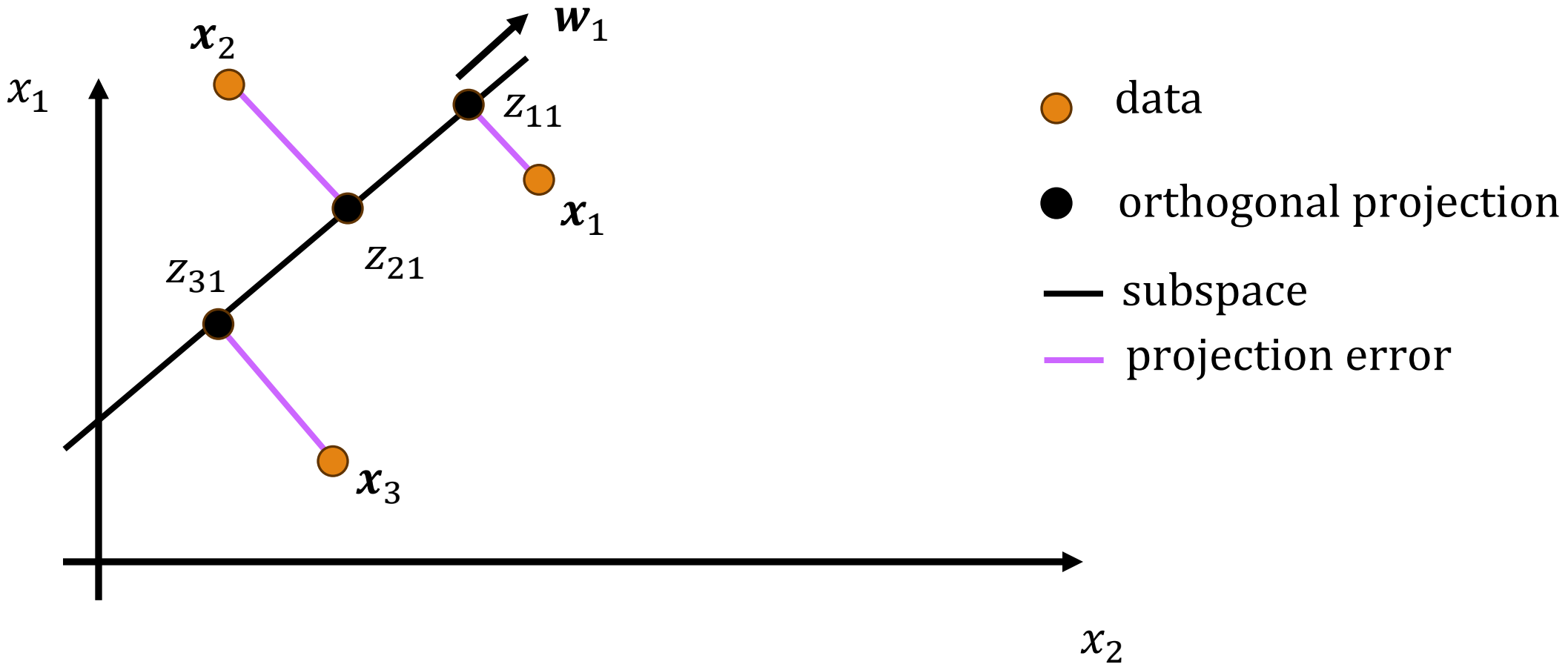


Figure source: C. M. Bishop, Pattern Recognition and Machine Learning, Springer NY, 2006

Principal Components Analysis (PCA)

$$L(\boldsymbol{w}_1, \boldsymbol{z}_1) = \frac{1}{N}\sum_{i=1}^{N}(\boldsymbol{x}^T{}_i\boldsymbol{x}_i - 2\, z_{i1}\boldsymbol{w}^T{}_1\boldsymbol{x}_i + z^2{}_{i1}).$$

$\frac{\partial L(\boldsymbol{w}_1,\boldsymbol{z}_1)}{\partial \boldsymbol{z}_1} = 0$, We get $\frac{1}{N}$ (-2 $\boldsymbol{w}^T{}_1\boldsymbol{x}_i$ + 2 z_{i1}) = 0, $\quad z_{i1} = \boldsymbol{w}^T{}_1\boldsymbol{x}_i$

Substitute $z_{i1} = \boldsymbol{w}^T{}_1\boldsymbol{x}_i$ to $L(\boldsymbol{W}, \boldsymbol{z}_1)$, * Now we need to find direction w_1

$$L(\boldsymbol{w}_1, \boldsymbol{z}^*{}_1(\boldsymbol{w}) = \frac{1}{N}\sum_{i=1}^{N}(\boldsymbol{x}^T{}_i\boldsymbol{x}_i - 2\boldsymbol{w}^T{}_1\boldsymbol{x}_i\boldsymbol{w}^T{}_1\boldsymbol{x}_i + z^2{}_{i1}) = \frac{1}{N}\sum_{i=1}^{N}(\boldsymbol{x}^T{}_i\boldsymbol{x}_i - z^2{}_{i1}).$$

$$= \text{constant} + \frac{1}{N}\sum_{i=1}^{N}(-z^2{}_{i1}).$$

empirical covariance matrix

$$L(\boldsymbol{w}_1) = \frac{1}{N}\sum_{i=1}^{N} -(\boldsymbol{w}^T{}_1\boldsymbol{x}_i)^2 = -\frac{1}{N}\sum_{i=1}^{N}\boldsymbol{w}^T{}_1\boldsymbol{x}_i\ \boldsymbol{x}^T{}_i\boldsymbol{w}_1 = -\boldsymbol{w}^T{}_1\widehat{\boldsymbol{\Sigma}}\,\boldsymbol{w}_1.$$

before calculating PC directions center dataset at zero

Empirical covariance matrix $\widehat{\boldsymbol{\Sigma}} = \frac{1}{N}\sum_{i=1}^{n}(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^T(\boldsymbol{x}_i - \overline{\boldsymbol{x}})$ and we assumed that data is centred.

Principal Components Analysis (PCA)

optimize loss w.r.t w_1 direction

use Lagrange multipliers

result is eigenvalue of covariance matrix

$$L(\boldsymbol{w}_1) = -\boldsymbol{w}^T{}_1 \widehat{\boldsymbol{\Sigma}}\, \boldsymbol{w}_1.$$

- We optimize this by $\|\boldsymbol{w}_1\| \to \infty$. So, we impose constrain $\boldsymbol{w}_1{}^T \boldsymbol{w}_1 = 1$.
- We optimize following.

$$\tilde{L}(\boldsymbol{w}_1) = \boldsymbol{w}^T{}_1 \widehat{\boldsymbol{\Sigma}}\, \boldsymbol{w}_1 - \lambda_1 (\boldsymbol{w}_1{}^T \boldsymbol{w}_1 - 1).$$

λ_1 Lagrange multiplier

$$\frac{\partial \tilde{L}(\boldsymbol{w}_1)}{\partial \boldsymbol{w}_1} = 0 = 2\, \widehat{\boldsymbol{\Sigma}}\, \boldsymbol{w}_1 - 2\, \lambda_1 \boldsymbol{w}_1.$$

$$\widehat{\boldsymbol{\Sigma}}\, \boldsymbol{w}_1 = \lambda_1 \boldsymbol{w}_1.$$

$\boldsymbol{w}_1$ eigenvector of the covariance matrix $\widehat{\boldsymbol{\Sigma}}$

Multiplying $\boldsymbol{w}^T{}_1$ and using $\boldsymbol{w}_1{}^T \boldsymbol{w}_1 = 1$, we get $\boldsymbol{w}^T{}_1\, \widehat{\boldsymbol{\Sigma}}\, \boldsymbol{w}_1 = \lambda_1$. *and* $L(\boldsymbol{w}_1) = -\boldsymbol{w}^T{}_1 \widehat{\boldsymbol{\Sigma}}\, \boldsymbol{w}_1$.

- To maximize this quantity $\boldsymbol{w}^T{}_1 \widehat{\boldsymbol{\Sigma}}\, \boldsymbol{w}_1$ (which corresponds to minimizing the loss), we select the eigenvector associated with the largest eigenvalue (**first principal component**).

Lagrange multiplier | Lagrange multiplier exemples (mit.edu) | eigenvector

Steps:
1. Calculate Σ
2. Get eigenvalues/vectors

3. Select largest k eigenvalues → eigenvectors are the principal components

Principal Components Analysis (PCA)

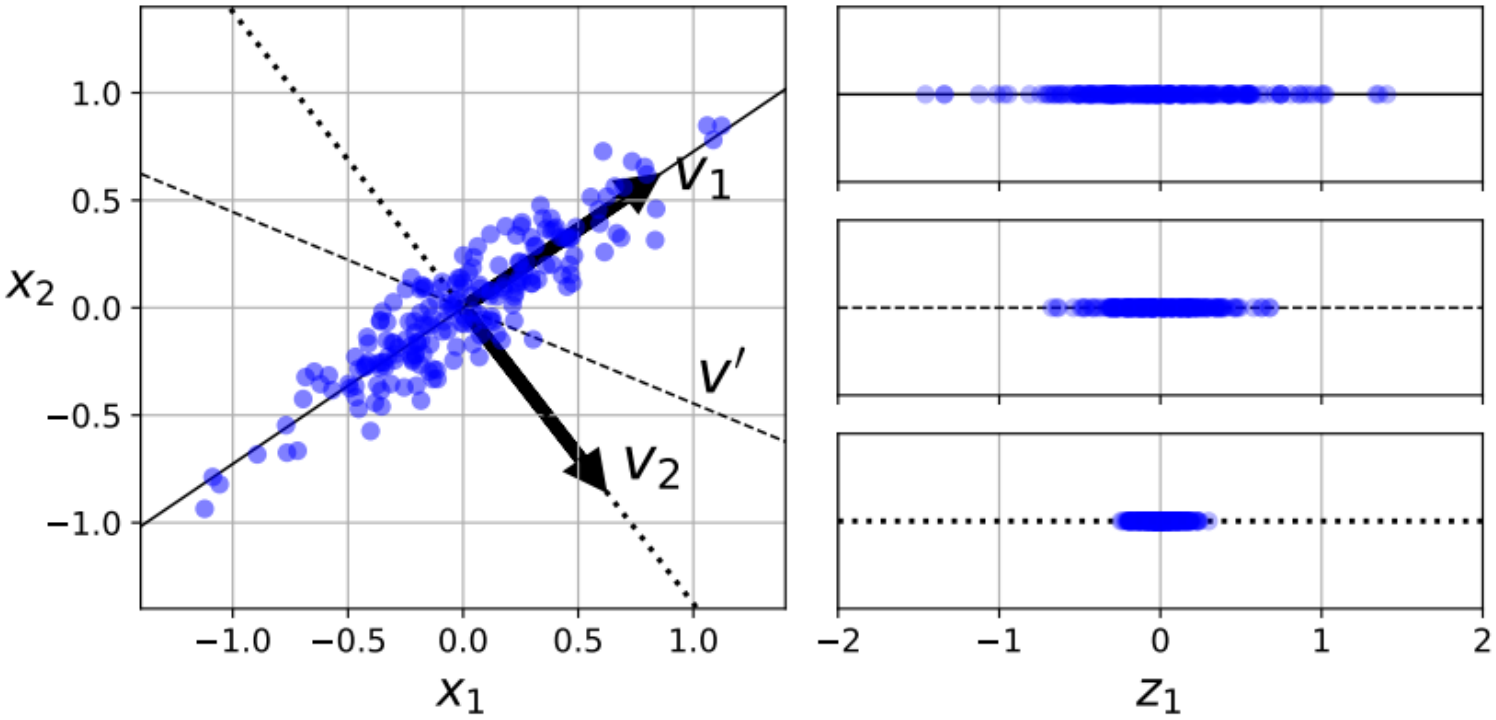
- Consider another direction $\boldsymbol{w}_2$, where $\boldsymbol{w}^T{}_1\, \boldsymbol{w}_2 = 0$ and $\boldsymbol{w}^T{}_2\, \boldsymbol{w}_2 = 1$.
- Now, $L(\boldsymbol{w}_1, \boldsymbol{z}_1, \boldsymbol{z}_2) \triangleq$
- Same way we can show that $\widehat{\boldsymbol{\Sigma}}\, \boldsymbol{w}_2 = \lambda_2 \boldsymbol{w}_2$ and $\boldsymbol{w}\boldsymbol{T}_2\, \widehat{\boldsymbol{\Sigma}}\, \boldsymbol{w}_2 = \lambda_2$.
- Similarly, $\boldsymbol{W} = \boldsymbol{V}_L \in \mathbb{R}^{dxl}$ where $\boldsymbol{V}_L$ contains the l eigenvectors with largest eigenvalues of the empirical covariance matrix $\widehat{\boldsymbol{\Sigma}}$.

Since Σ is symmetric eigenvectors are $\perp$

$$\boldsymbol{W} = \begin{bmatrix} \boldsymbol{w}_1 \\ \vdots \\ \boldsymbol{w}_l \end{bmatrix}$$



Principal Components Analysis (PCA)



- First principal component (v_1) (maximizes the variance of the projection)
- Second principal component (v_2) (orthogonal to v_1)
- Another vector between v_1 and v_2.

Variance of the points projected onto different 1d vectors*.

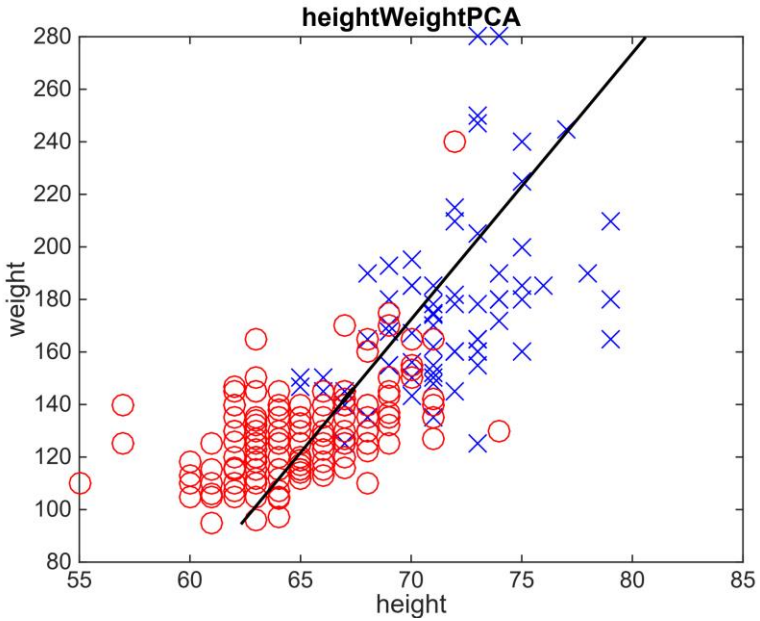
Code (pca_projected_variance.ipynb)

*Figure from Géron, Aurélien. Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow. " O'Reilly Media, Inc.", 2022.

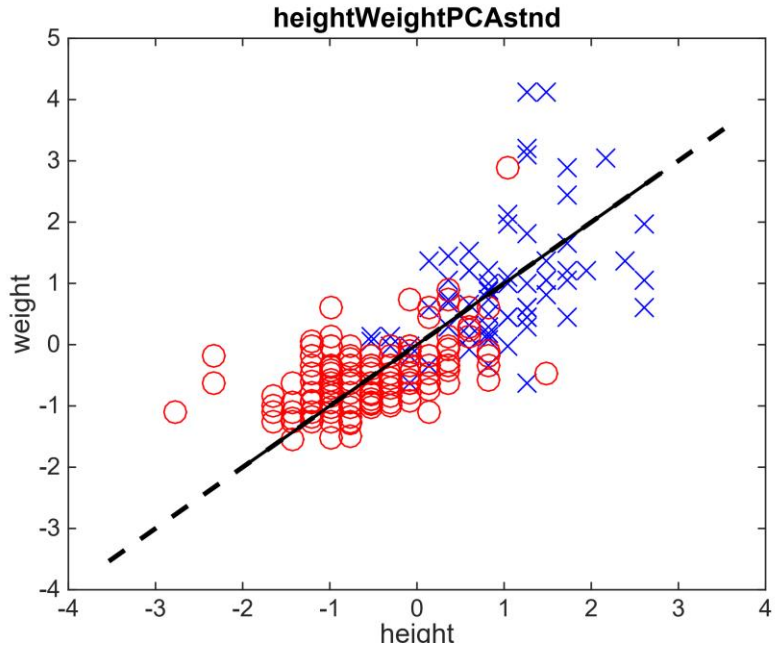
Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.

PCA-Covariance matrix vs correlation matrix

- Instead of using the covariance matrix, it's more advantageous to work with the correlation matrix in eigen decomposition of data.
- This prevent PCA from being influenced by directions with high variance primarily due to differences in measurement scales.



Vertical axis has a larger range than the horizontal one, leading to an "unnatural" appearance of the first principal component



PCA after standardizing the data, which is equivalent to using the correlation matrix

covariance matrix

dependent on scale

$$\mathrm{Cov}(X,Y) = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})$$

correlation matrix

independent of feature scales

$$r_{XY} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

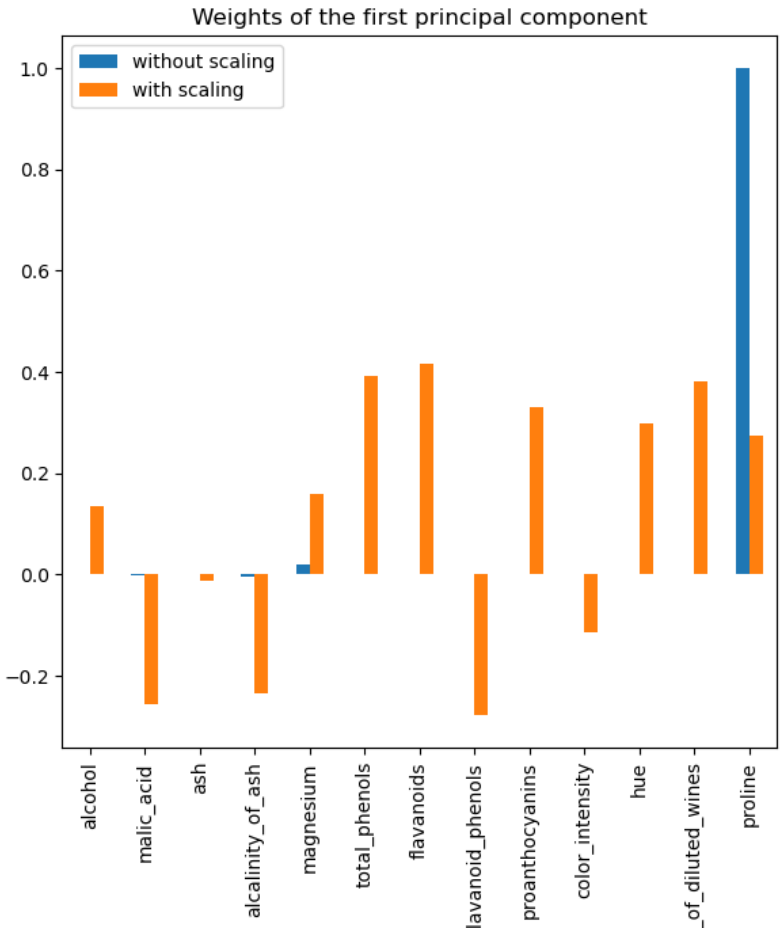
gets rid of feature scaling

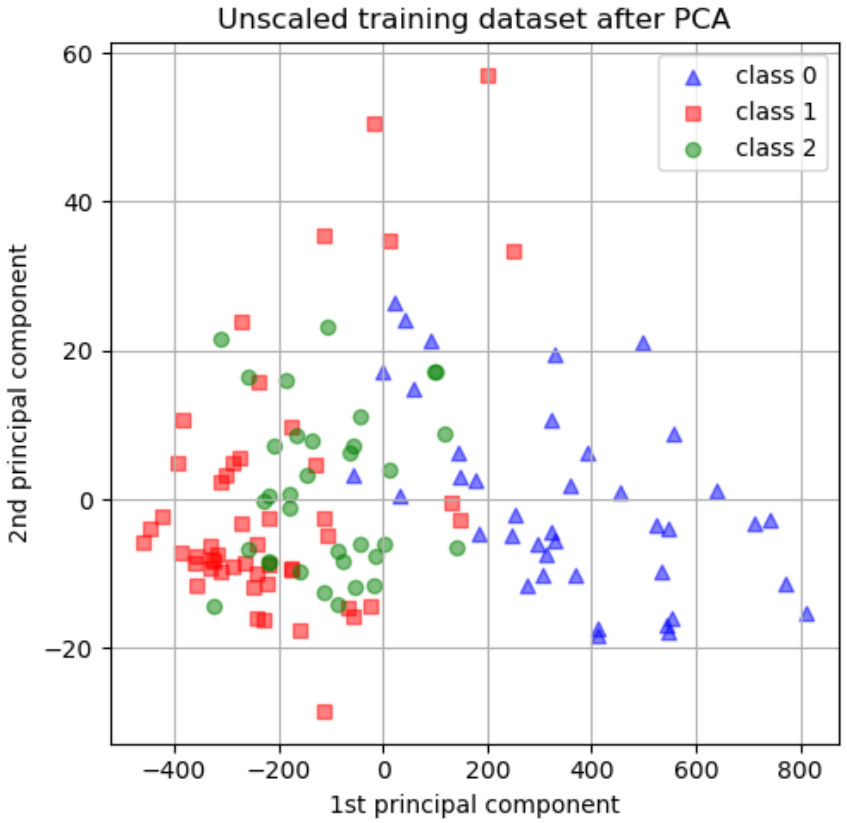
σ_x , σ_y –standard deviations

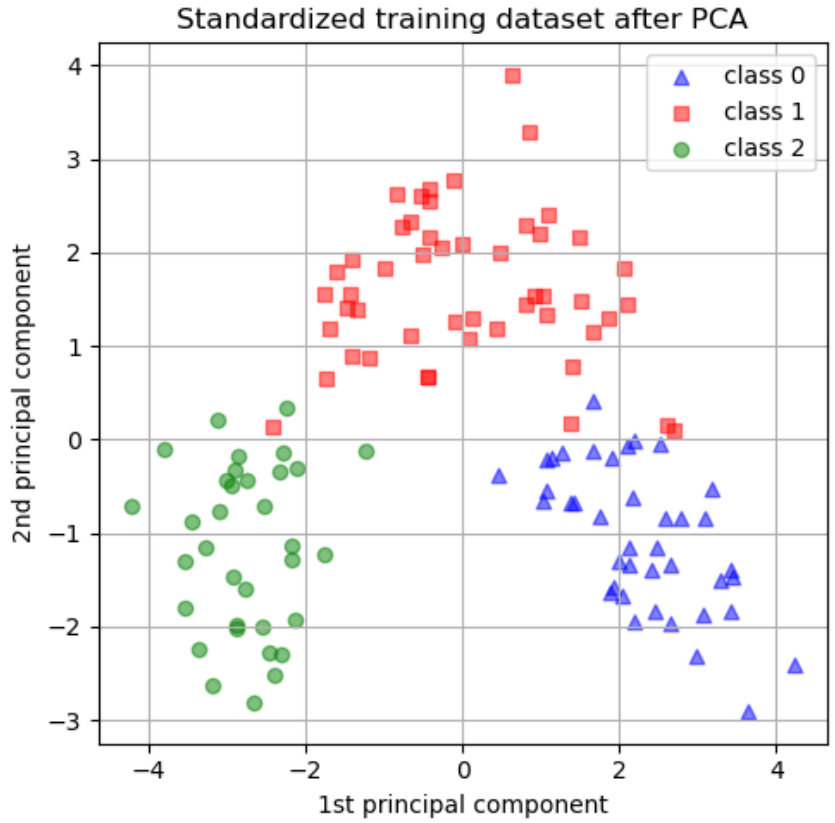
Code (*pcaStandardization*.ipynb)

Figure from Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.

PCA- Feature scaling







- Test accuracy for the unscaled PCA 35.19%.
- Test accuracy for the standardized data with PCA 96.30%.
- Scaling improves performance of the model. → not always true
- This is not always happening*.

Source: Sklearn PCA scaling_importance.

Code: Sklearn PCA scaling_importance.

*If the variables with lower scales are not informative or predictive in a given model, scaling those features may lead to a decrease in model performance. This is because when you scale the features, noisy or uninformative features that were previously in a lower scale may now contribute more to the prediction.

dimension of a data sample

no. of data samples

Principal Components Analysis (PCA)

- High dimensional data $\boldsymbol{x} \in \mathbb{R}^D$
- If $D \gg N$. It is faster to use N x N Gram matrix ($\boldsymbol{X}^T\boldsymbol{X}$) than covariance matrix of size D x D. Here, X is data matrix of size D x N. (e.g., 1000 images of 1000 x 1000 pixels size, N=1000, D=10^6).
- Suppose $\boldsymbol{U}$ an orthogonal matrix containing the eigenvectors of $\boldsymbol{X}^T\boldsymbol{X}$.
- From difinition: $(\boldsymbol{X}\,\boldsymbol{X}^T)\,\boldsymbol{U} = \boldsymbol{U}\,\boldsymbol{\Lambda}$. Here, $\boldsymbol{\Lambda}$ contains corresponding eigenvalues.

This allows us to use the kernel trick.

Multiply with $\boldsymbol{X}^T$, we get : $\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{X}^T)\,\boldsymbol{U} = \boldsymbol{X}^T\,\boldsymbol{U}\,\boldsymbol{\Lambda}$.

$$(\boldsymbol{X}^T\boldsymbol{X})(\boldsymbol{X}^T\boldsymbol{U}) = (\boldsymbol{X}^T\,\boldsymbol{U})\,\boldsymbol{\Lambda}$$

eigen vectors of $(\boldsymbol{X}^T\boldsymbol{X})$ are $\boldsymbol{V} = (\boldsymbol{X}^T\boldsymbol{U})$. This is not normalized (to be unit vector)*.

The normalized eigenvectors are given by $\boldsymbol{V} = \boldsymbol{X}^T\boldsymbol{U}\boldsymbol{\Lambda}^{-\frac{1}{2}}$

*$\|\boldsymbol{v}_i\| = \boldsymbol{u}_i{}^\top \boldsymbol{X}\boldsymbol{X}^T\boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i{}^\top\boldsymbol{u}_i = \lambda_i$.

Kernel PCA

- Sample covariance matrix in feature space* $\mathbf{S}_{\phi} = \frac{1}{N} \sum_{i=1}^{N} \phi(\boldsymbol{x}_i)\phi(\boldsymbol{x}_i)^{\boldsymbol{T}} = \frac{1}{N} \sum_{i=1}^{n} \phi_i \phi_i^{\boldsymbol{T}}$.
- Gram matrix **K = XX**^{T}.
- $\boldsymbol{V}_{kPCA} = \boldsymbol{\phi}^{\boldsymbol{T}} \boldsymbol{U} \boldsymbol{\Lambda}^{-\frac{1}{2}}$, Here, $\boldsymbol{U}$ and $\boldsymbol{\Lambda}$ contain the eigenvectors and eigenvalues of **K****.
- Projection of test data $\boldsymbol{x}_{test}$ to feature space

$$\phi_{test} \boldsymbol{V}_{kPCA} = \phi_{test} \boldsymbol{\phi}^{\boldsymbol{T}} \boldsymbol{U} \boldsymbol{\Lambda}^{-\frac{1}{2}} = \boldsymbol{k}^{T}_{test} \boldsymbol{U} \boldsymbol{\Lambda}^{-\frac{1}{2}}, \text{ with } \boldsymbol{k}_{test} = [k(\boldsymbol{x}_{test}, \boldsymbol{x}_1), \ldots k(\boldsymbol{x}_{test}, \boldsymbol{x}_N)].$$

- If features are not centered ***, replace $\phi(\boldsymbol{x}_i)$ with $\tilde{\phi}(\boldsymbol{x}_i)$

$$\tilde{\phi}(\boldsymbol{x}_i) = \phi(\boldsymbol{x}_i) - \frac{1}{N} \sum_{i=1}^{N} \phi(\boldsymbol{x}_i).$$

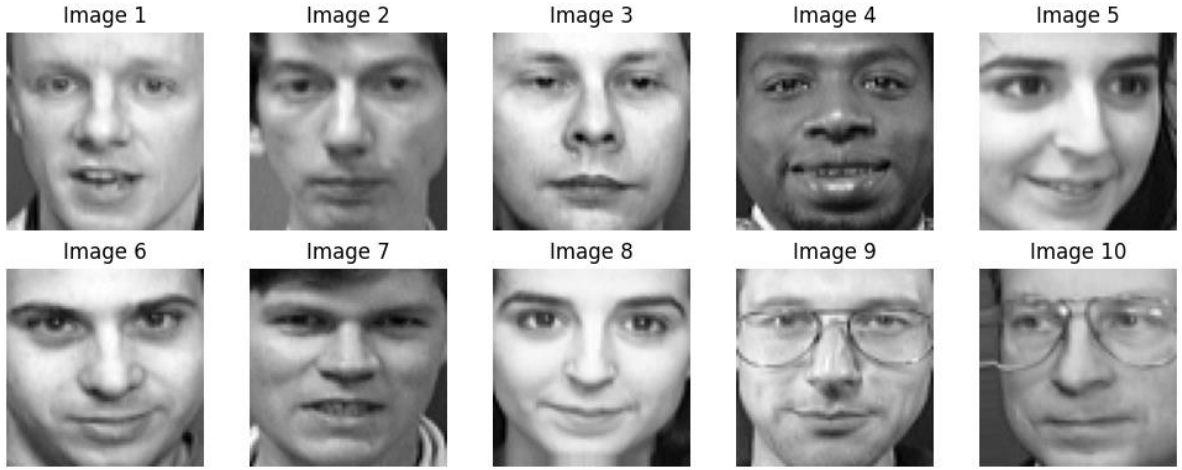
*Assume that features are centered

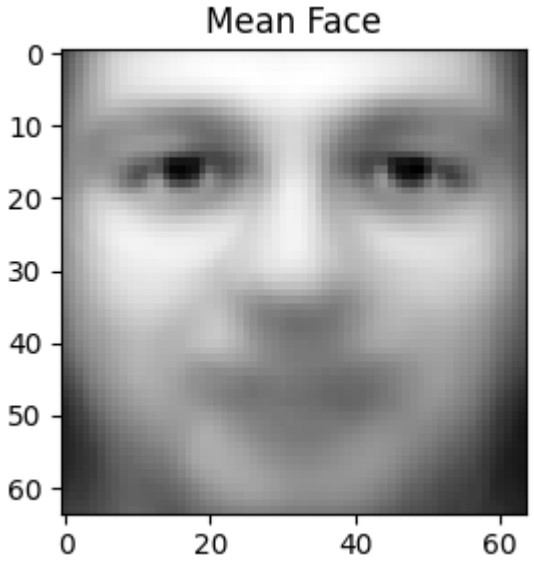
** we may not able to calculate $\boldsymbol{V}_{kPCA}$ since ϕ potentially infinite dimensional. However, projection of test data to feature space can be computed.

*** we cannot simply subtract off the mean in feature space, since it may be infinite dimensional .

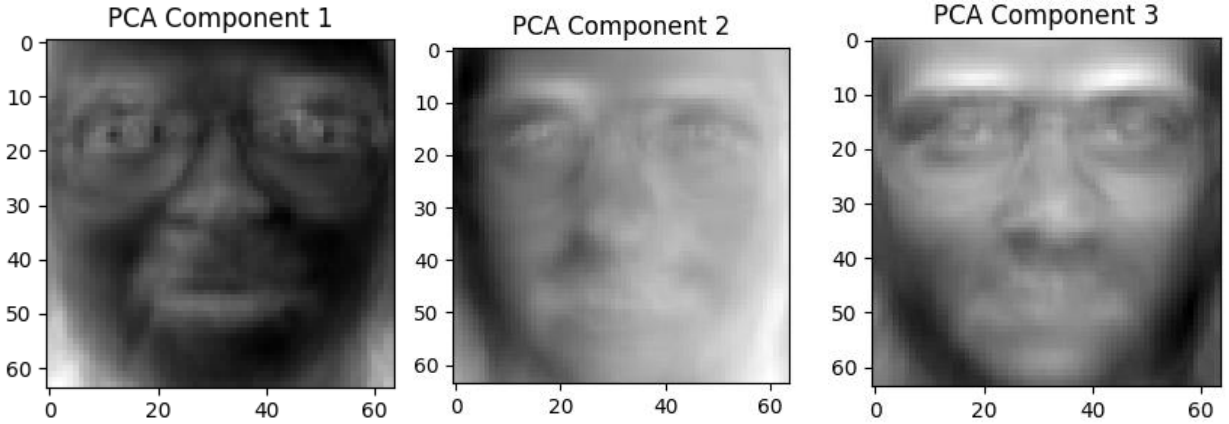
PCA

Some images from the Olivetti face database (64 × 64 grayscale images)

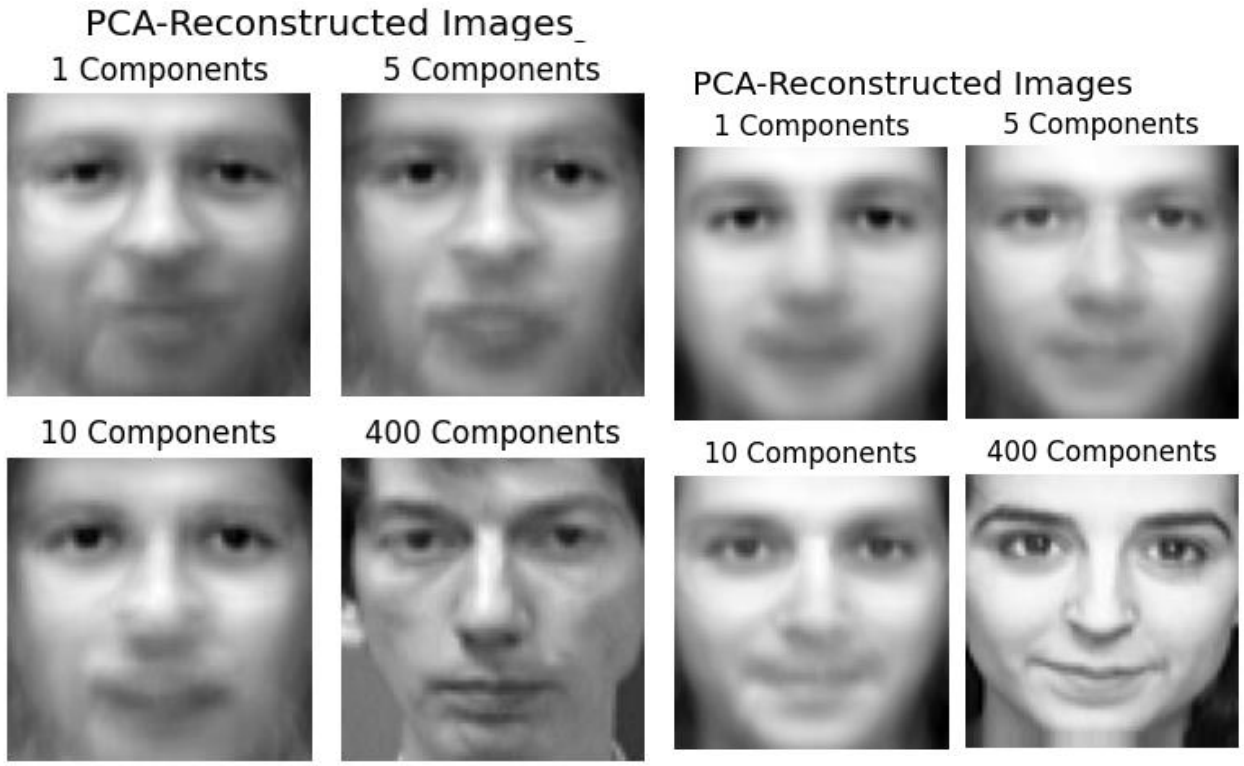




4096 features



Individual PCA components



PCA reconstruction

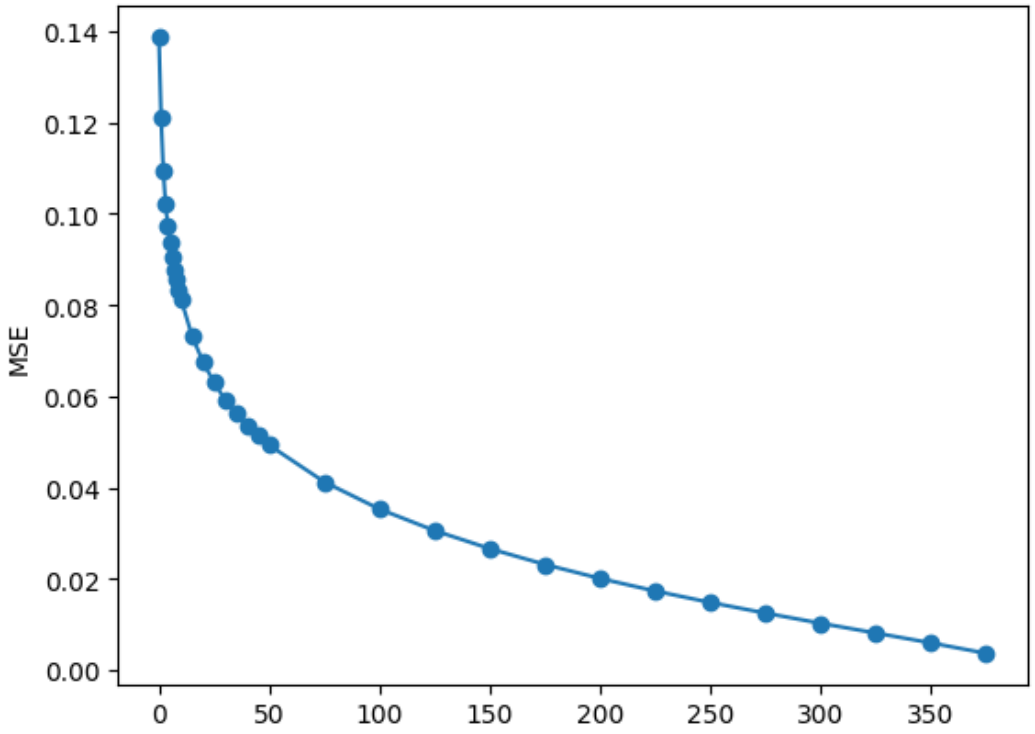
Code (*pcaImageDemo.ipynb*)

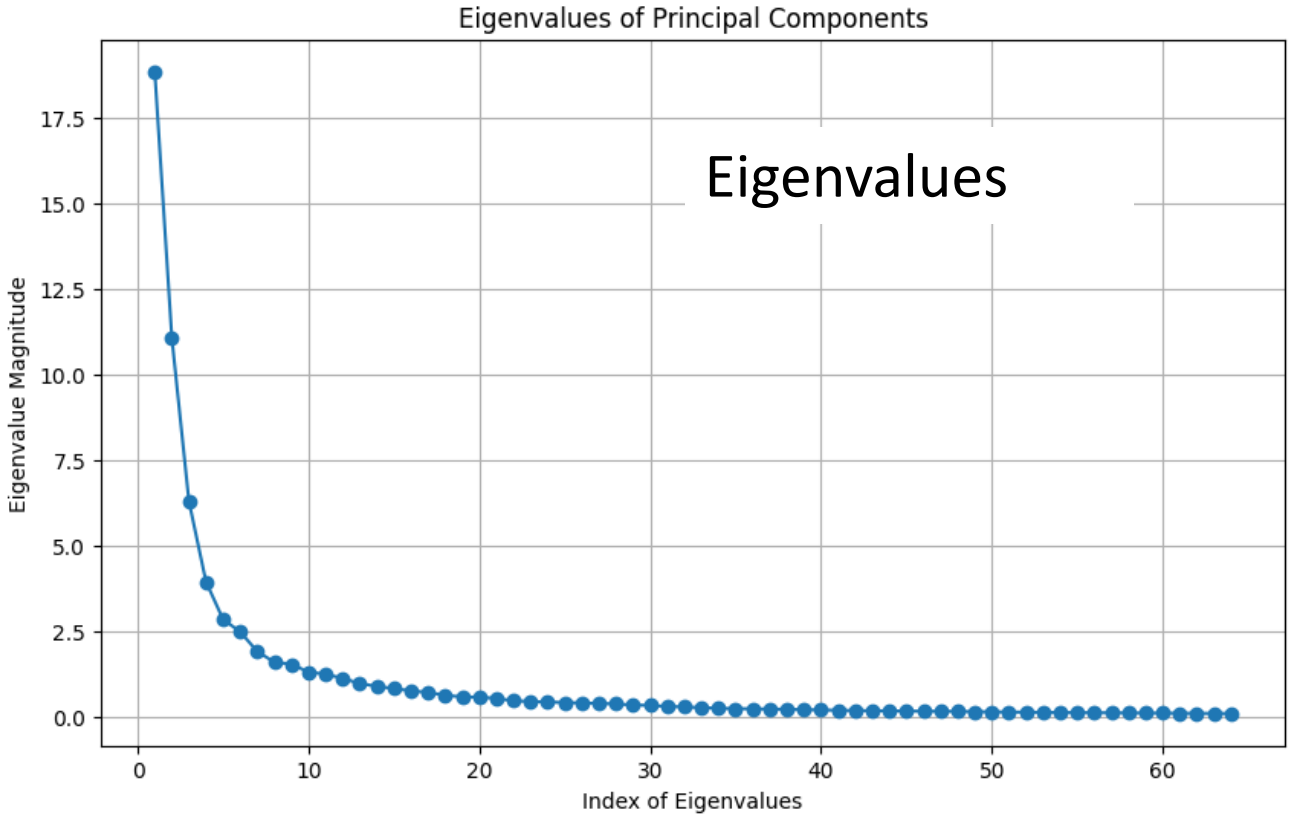
Figure from Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.

PCA

- *Olivetti face database*

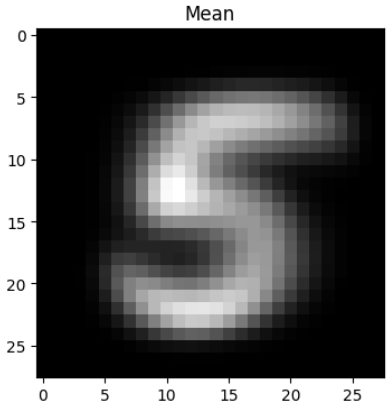
Reconstructed error





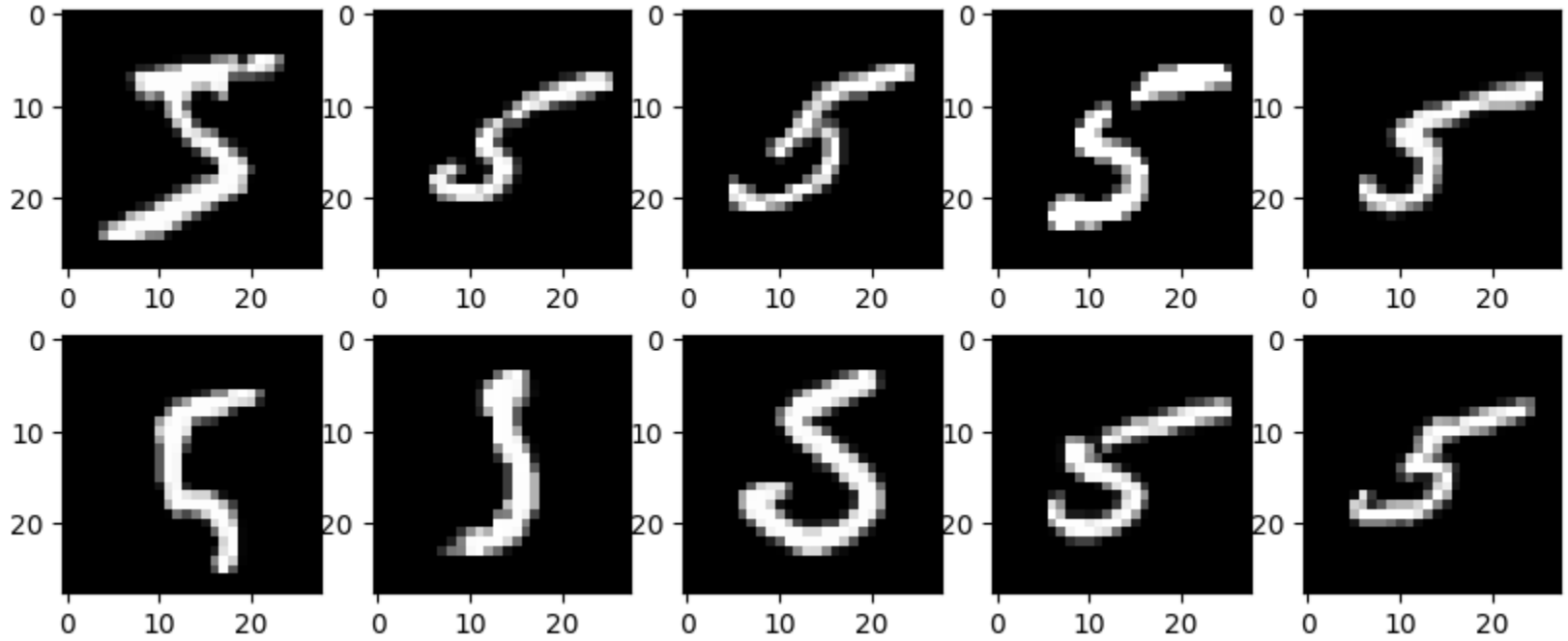
Code (*pcaImageDemo.ipynb*) from Kevin P. Murphy, Machine Learning: A Probabilistic Perspective.

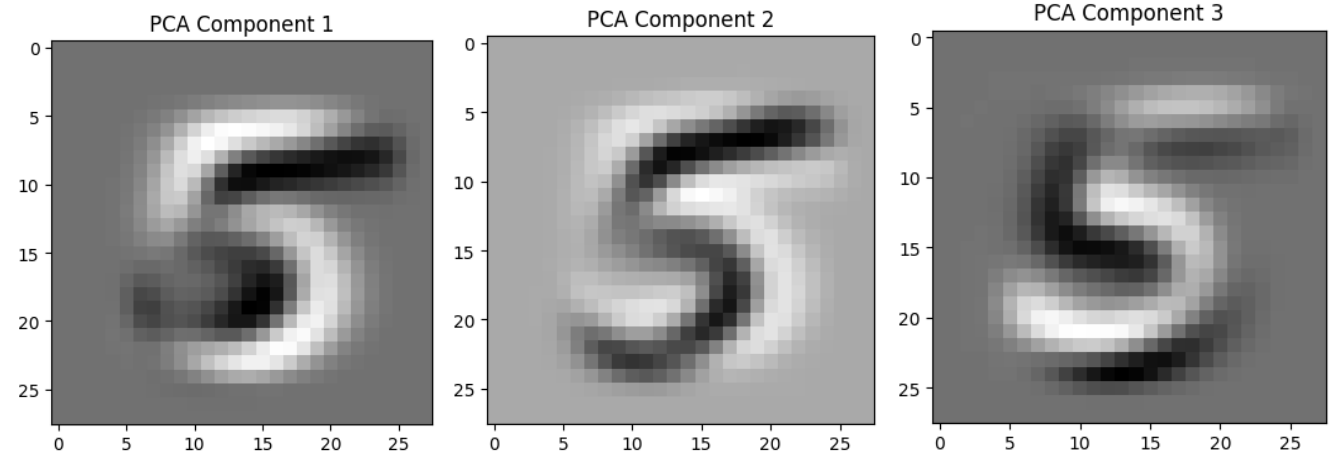
PCA



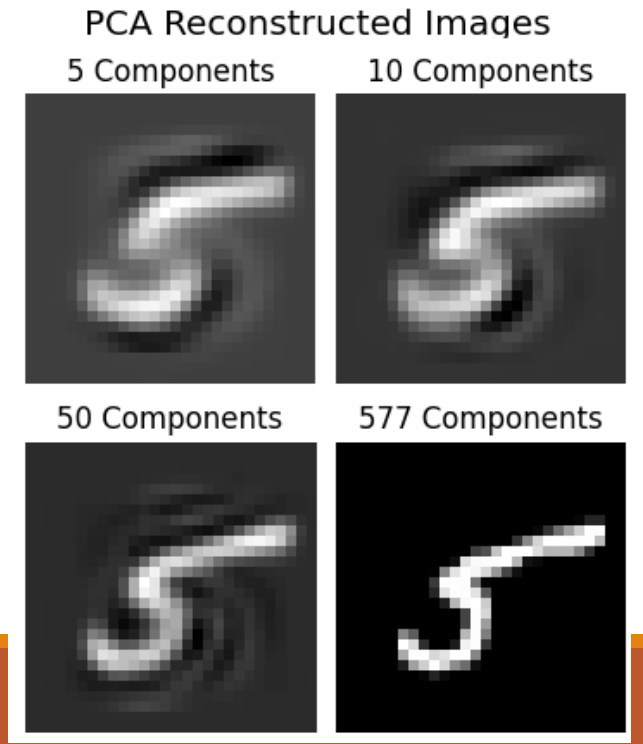
28 × 28 grayscale images

784 features



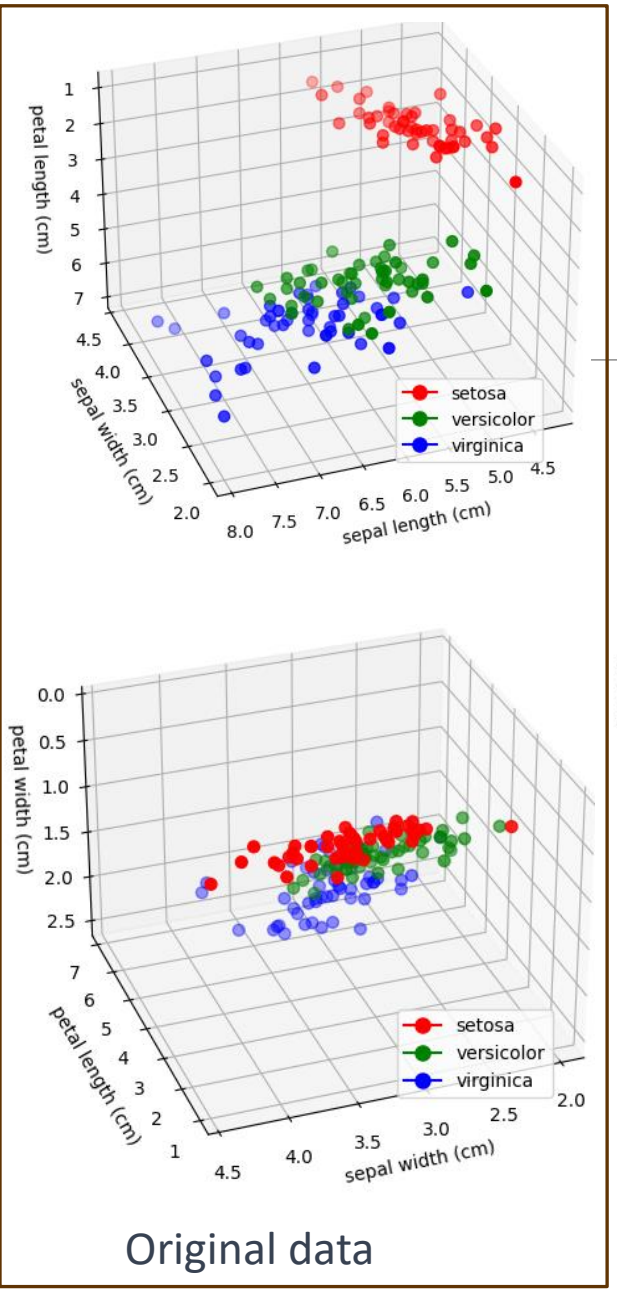


Individual PCA components



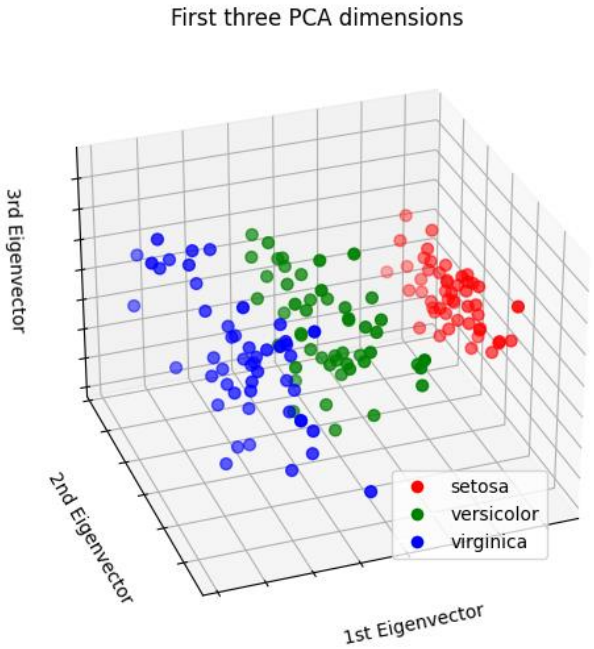
PCA reconstruction

PCA-Iris dataset

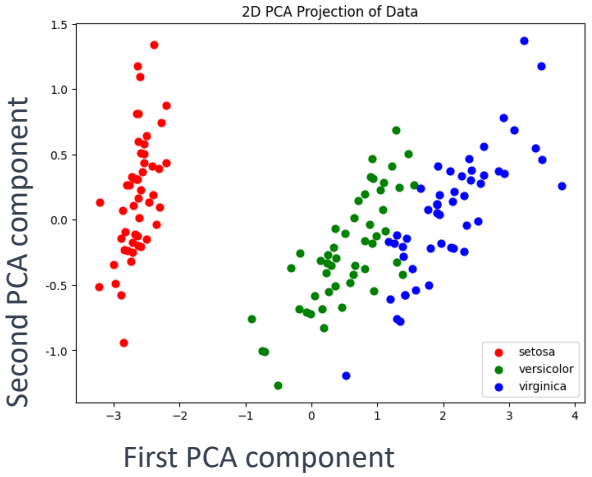


Original data

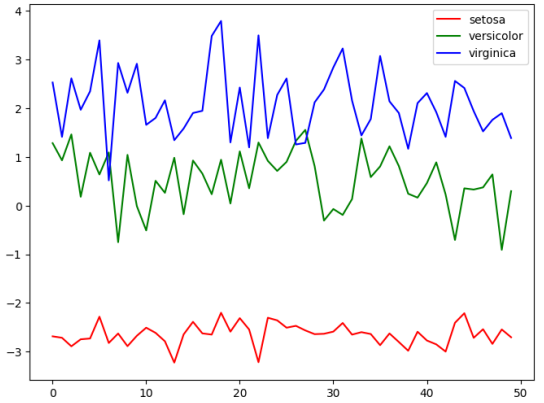
Features: sepal length, sepal width, petal length, petal width



PCA transformation maximizes the variance



First PCA component



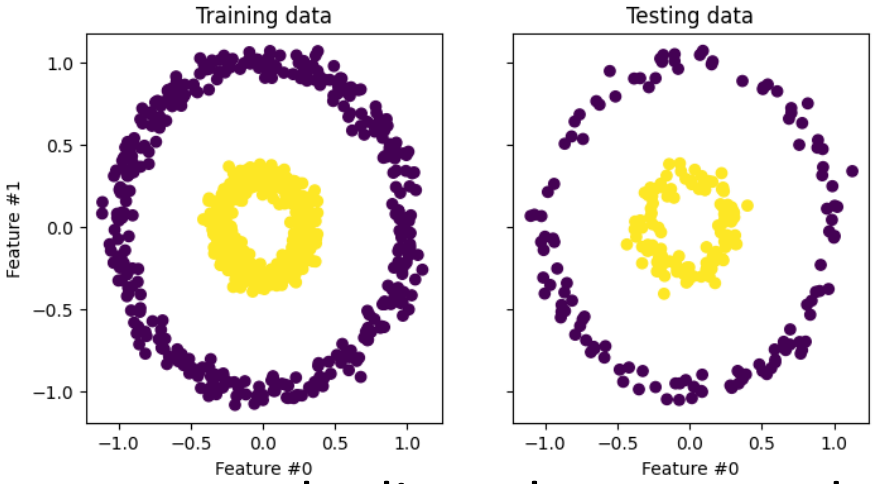
Using PCA, it becomes possible to identify each species using only the first feature, represented by the first eigenvalue.

Source: sklearn iris dataset.

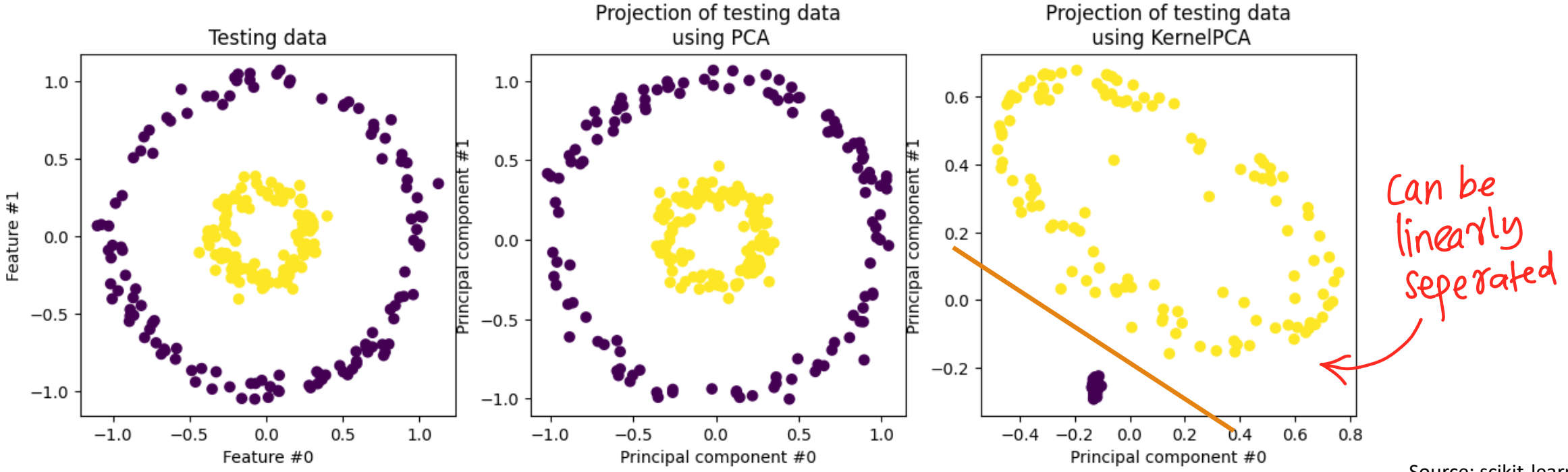
Code: sklearn iris dataset.

KernelPCA

- PCA performs a linear transformation on the data.
- Kernels enable non-linear projections.
- Same class samples exhibit greater proximity compared to samples from different classes.



cannot be linearly separated

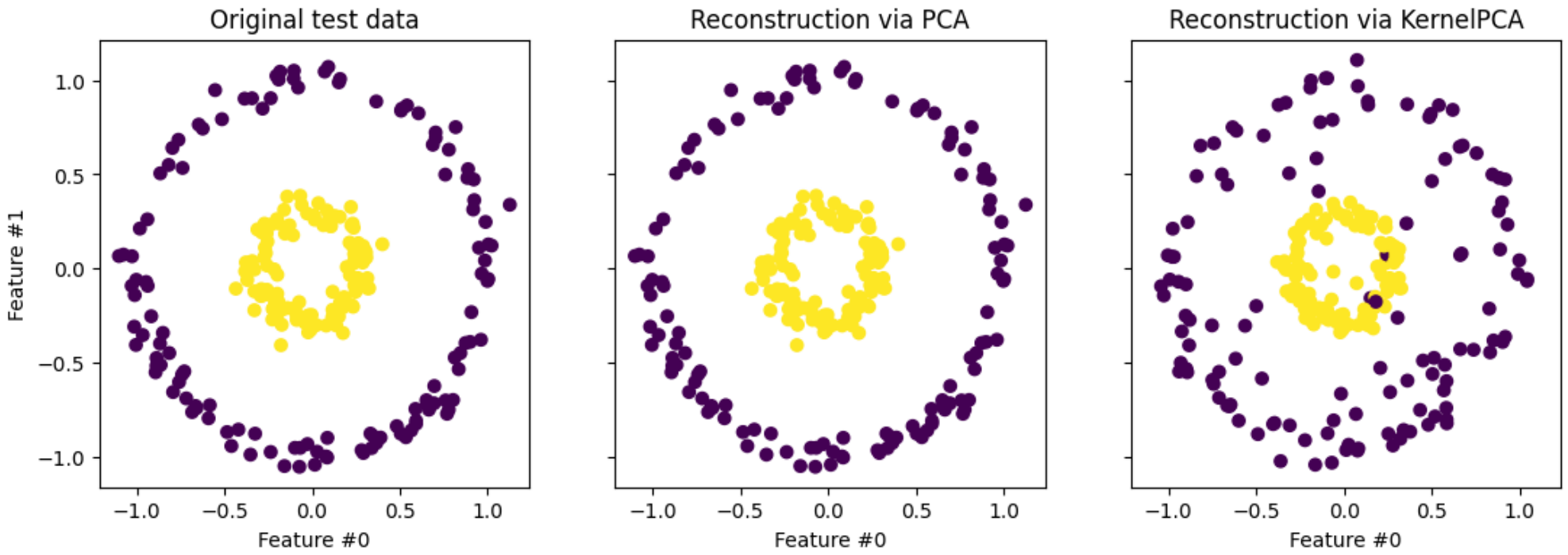


Source: scikit-learn kernel pca

Code: Kernel PCA — scikit-learn

KernelPCA: reconstruction

- PCA provides extract reconstruction (if no. of PCA component is same as no. of features).
- KernelPCA mapping back to input space is not straightforward process.



Source: scikit-learn kernel pca

Code: Kernel PCA — scikit-learn

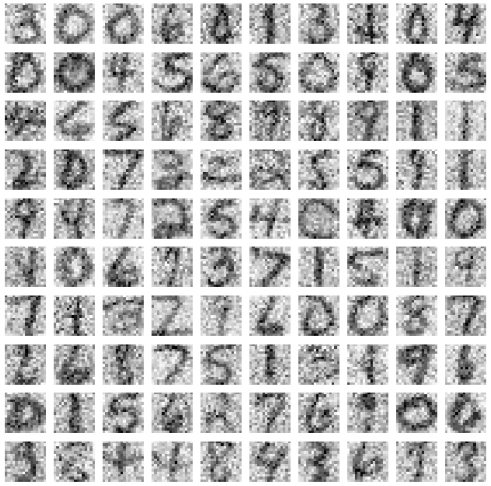
Image denoising

- Trained on noisy train images.
- For this example, PCA has a lower MSE compared to Kernel PCA.
- Qualitative analysis may not prefer PCA as denoising with Kernel PCA depends on parameter choices.
- Kernel PCA's provides smoother images.

Uncorrupted train images



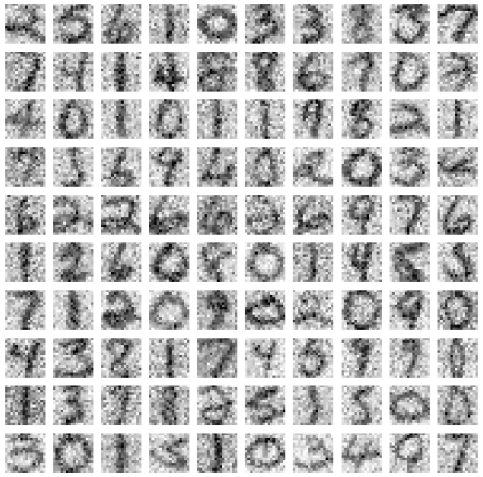
Noisy train images
MSE: 0.06



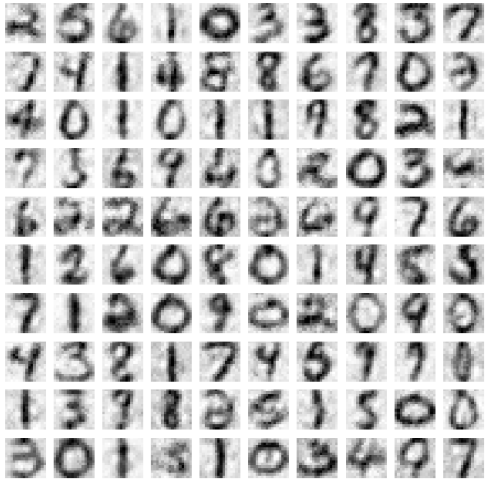
Uncorrupted test images



Noisy test images
MSE: 0.06



PCA reconstruction
MSE: 0.01



Kernel PCA reconstruction
MSE: 0.03



Source:digits_denoising.

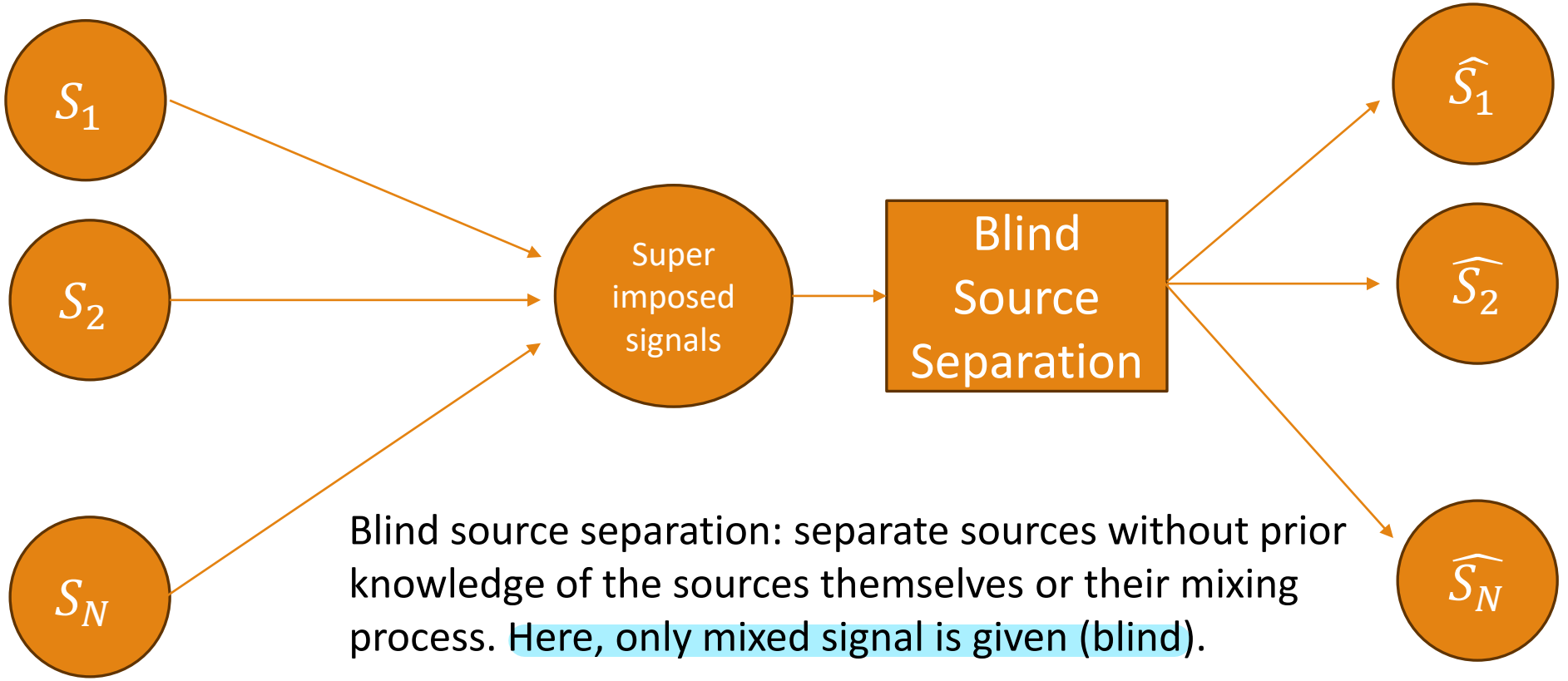
Code: Kernel PCA — scikit-learn

More examples

- Faces dataset decompositions

- A demo of K-Means clustering on the handwritten digits data

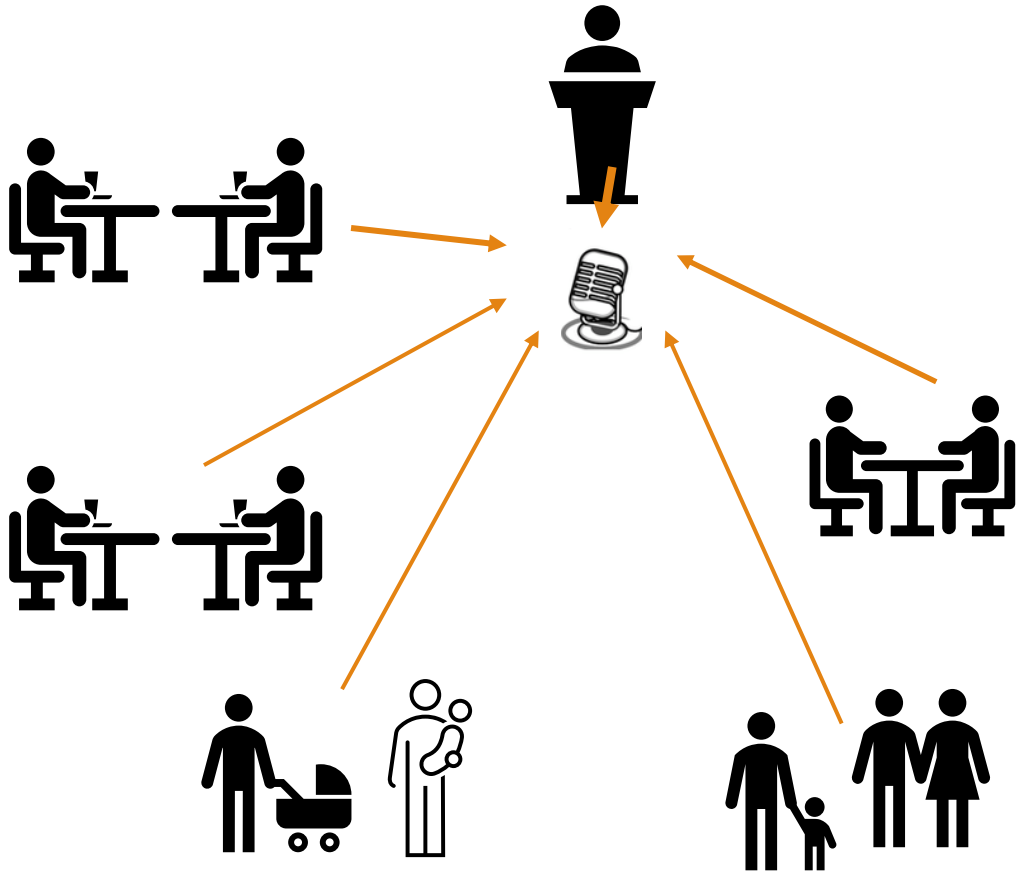
Independent Component Analysis (ICA)



Blind source separation: separate sources without prior knowledge of the sources themselves or their mixing process. Here, only mixed signal is given (blind).

Independent Component Analysis (ICA)

- **Cocktail party problem**
- Scenario: While main speaker delivering the speech, multiple people talking simultaneously at a party, recorded by multiple microphones.
- Challenge: Each microphone records a linear combination of the individual voices.
- Solution: ICA aims to unmix these signals to recover the original sources (Main Speaker).



Independent Component Analysis (ICA)

- Goal of ICA is to find a linear transformation of the original data in such a way that the resulting components are as statistically independent from each other as possible.
- Usually, ICA is not used for dimensionality reduction but for the separation of overlapped or mixed $\mathbf{non-Gaussian}$ signals.
- Linear mixture of N sources

sources

$$x_j = a_{j1}s_1 + a_{j2}s_2 + \ldots + a_{jN}s_N, \text{ for all } j.$$

examples (data)

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \cdot \\ \cdot \\ \cdot \\ x_M \end{bmatrix} \quad \boldsymbol{s} = \begin{bmatrix} s_1 \\ s_2 \\ \cdot \\ \cdot \\ \cdot \\ s_N \end{bmatrix} \quad \boldsymbol{A} = \begin{bmatrix} a_{11} & a_{12} & & a_{1N} \\ a_{21} & a_{22} & & a_{2N} \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & \cdots & \cdot \\ \cdot & \cdot & & \cdot \\ a_{M1} & a_{M2} & & a_{MN} \end{bmatrix}$$

challenge is to estimate A

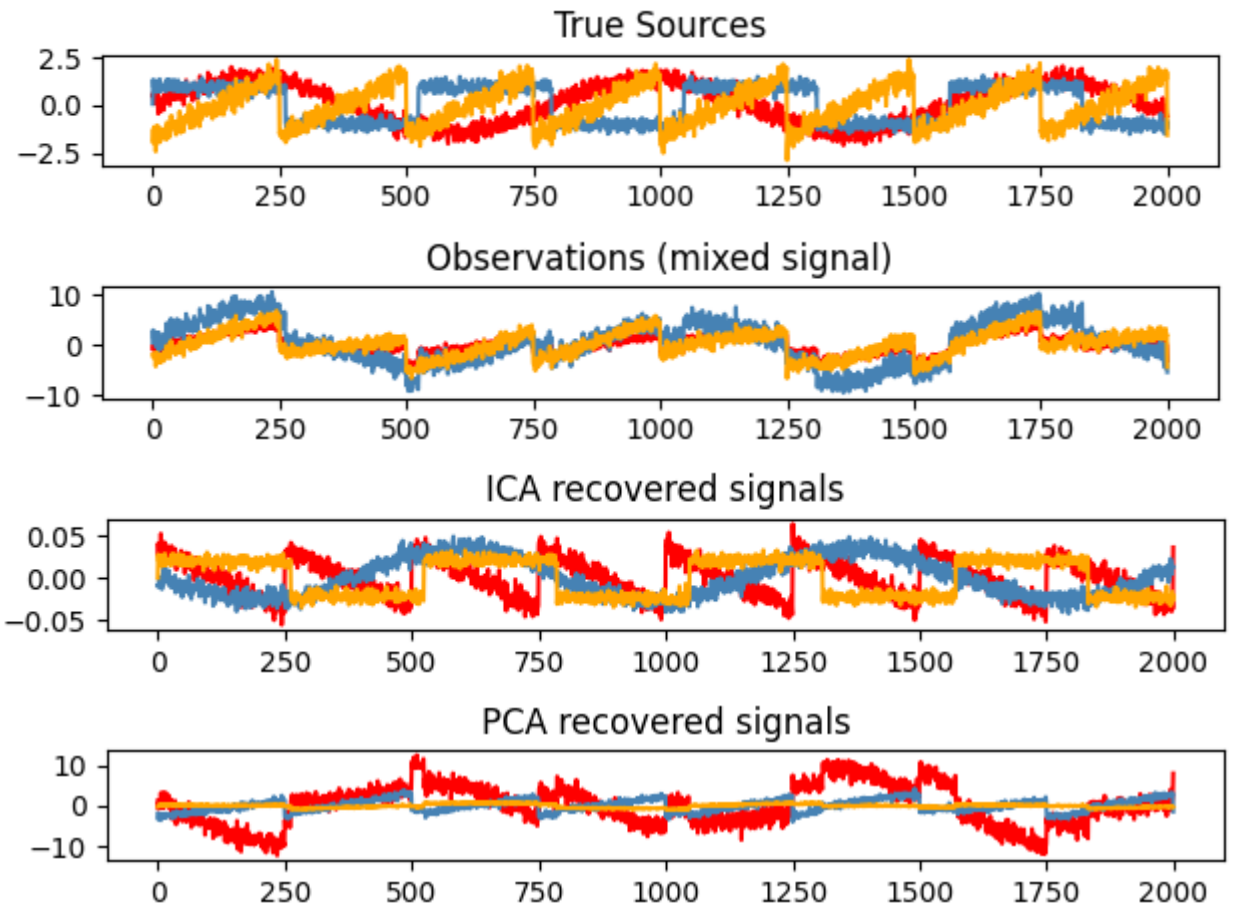
$\boldsymbol{x} = \boldsymbol{A}\,\boldsymbol{s}$. Mixing Model*

After estimating $\boldsymbol{A}$, We can obtain the independent component by

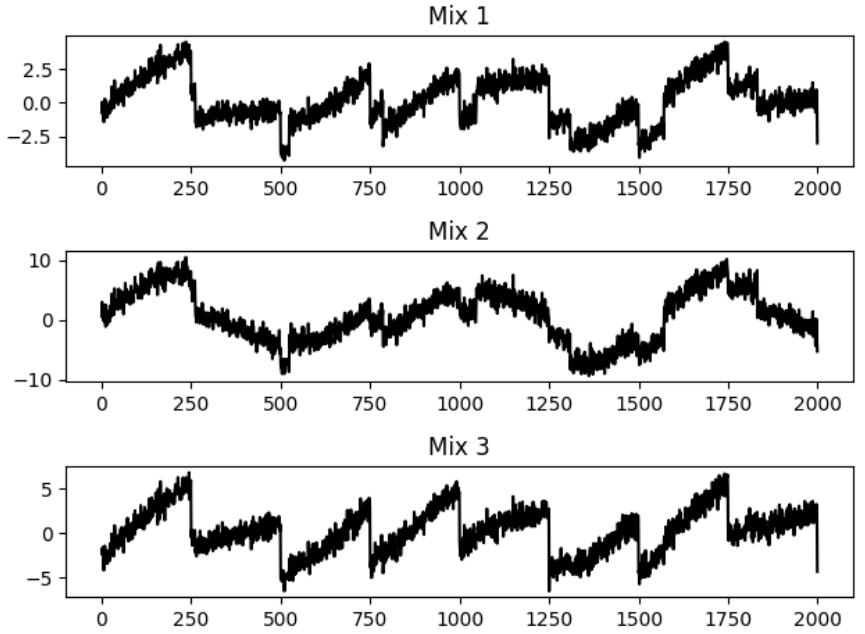
$\boldsymbol{s} = \boldsymbol{W}\,\boldsymbol{x}$.

noisy ICA model $\boldsymbol{x} = \boldsymbol{A}\,\boldsymbol{s} + \boldsymbol{n}$.

Independent Component Analysis (ICA)



$$A = \begin{bmatrix} 1.0 & 1.0 & 1.0 \\ 3.5 & 2.0 & 1.0 \\ 1.5 & 0.45 & 2.0 \end{bmatrix}$$



Sourse:sklearn ica_blind_source_separation.

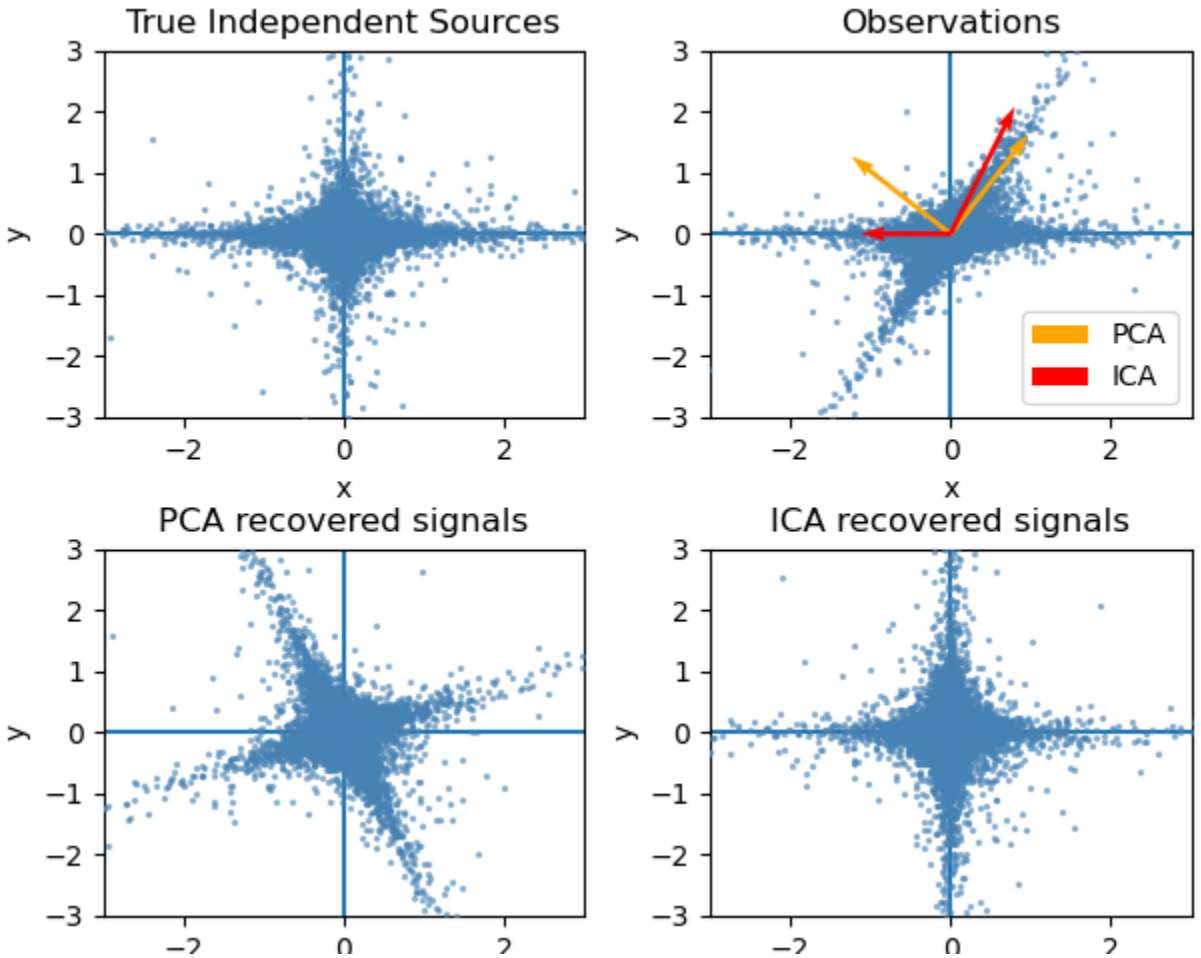
ICA

https://en.wikipedia.org/wiki/File:A-Local-Learning-Rule-for-Independent-Component-Analysis-srep28073-s3.ogv

ICA Applications

- Blind Source Separation
  - EEG or fMRI signal separation
  - Extracting specific audio sources from a sound mixture
  - Separating sources from remote sensing data
  - Remove artifact from ECG
- Image Processing
  - Feature extraction, or compression
  - Image denoising
  - Face recognition
- Clustering
- Classification
- Time series analysis
- Financial applications

PCA vs ICA



- Vectors are orthogonal in PCA.
- Vectors are not orthogonal in ICA.
- PCA does the compression, ICA not.
- ICA all the components are (equally) important
- PCA some components may be important than other.

Figure source:Sklearn ICA vs _PCA.

Source: https://www.cs.cmu.edu/~./bapoczos/other_presentations/ICA_26_10_2009.pdf

Multidimensional Scaling

we want similarities between data → Visualized on lower dimension

- Multidimensional Scaling (MDS)*: visualizing data similarities in a lower-dimensional space.
- Goal: To represent data points while preserving their pairwise distances or dissimilarities.
- MDS starts with a distance or dissimilarity matrix, quantifying relationships between data points.
- Projecting lower dimensional (2D or 3D) provides us to visualize data.





*Simplest approach to **manifold learning** is multidimensional scaling (MDS)

Multidimensional Scaling

Types of MDS

- Classical MDS
- Metric MDS
- Non-metric MDS

Multidimensional Scaling

Dissimilarity matrix $\boldsymbol{D} = \begin{bmatrix} d_{11} & d_{12} & \cdots & d_{1M} \\ d_{21} & d_{22} & \cdots & d_{2M} \\ \vdots & \vdots & \cdots & \vdots \\ d_{M1} & d_{M2} & \cdots & d_{MM} \end{bmatrix}$

different data points

- Given $\boldsymbol{D}$, objective is to find M vectors $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_M \in \mathbb{R}^N$ in low dimensional space such that

$$\|\boldsymbol{x}_i - \boldsymbol{x}_j\| \approx d_{ij} \ ,\forall \ \mathrm{i,j} \in 1,\ldots,\mathrm{M}.$$

dimension of low dimensional space

difference between two datapoints

similarities in the low-dimensional data space

similarities in the high-dimensional data space

- When N is 2 or 3, we are able to visualize $\boldsymbol{x}_i$.

Multidimensional Scaling

- Classical MDS

similarities in the low-dimensional data space

similarities in the high-dimensional data space

$$\text{Strain}\,(\boldsymbol{x}_1,\ldots,\boldsymbol{x}_M) = \left(\frac{\sum_{i\neq j=1,\ldots M.}\left(\|\boldsymbol{x}_i-\boldsymbol{x}_j\|-b_{ij}\right)^2}{\sum_{i\neq j=1,\ldots M} b^2{}_{ij}}\right)^{\frac{1}{2}}.$$

- Classical MDS assumes Euclidean distances (b_{ij} are Euclidean distances).

- It finds points in a lower-dimensional space that minimizes the stress/strain function directly through an eigenvalue decomposition, without the need for iterative optimization.

- Classical MDS is equivalent to PCA.

Multidimensional Scaling

- Metric MDS
- Let $\hat{d}_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|$.

$$\text{Strain}\,(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_M) = \left(\frac{\sum_{i \neq j = 1, \ldots M} (\hat{d}_{ij} - d_{ij})^2}{\sum_{i \neq j = 1, \ldots M} \hat{d}^2_{ij}} \right)^{\frac{1}{2}}$$

pairwise distances in the projected space (low-dimensional space)

pairwise dissimilarity/distance measure in the original feature space.

- Metric MDS uses an optimization process that minimizes the stress function using iterative algorithms.
- d_{ij} generalize it to allow for any dissimilarity measure.

not just Euclidean distance

SMACOF "Scaling by MAjorizing a COmplication Function". Can be used to solve this optimization problem (available in scikit-learn.)

Multidimensional Scaling

map distance to ranks

- **Non-metric** MDS

Let $\hat{d}_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|$.

$$\text{Strain}\,(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_M) = \left(\frac{\sum_{i \neq j = 1, \ldots M} \left(\hat{d}_{ij} - f(d_{ij})\right)^2}{\sum_{i \neq j = 1, \ldots M} \hat{d}^2{}_{ij}} \right)^{\frac{1}{2}}$$

- Instead of using distances directly, we consider the rankings of points.
- A monotonic function is used to map distances to ranks.
- Function f can be linear, as an example parametric monotonic function e.g., $f(d_{ij}) = a + b\, d_{ij}$

example monotonic function

- In non-metric MDS, the dissimilarities (d_{ij}) are **qualitative**, ordinal, or non-metric in nature.

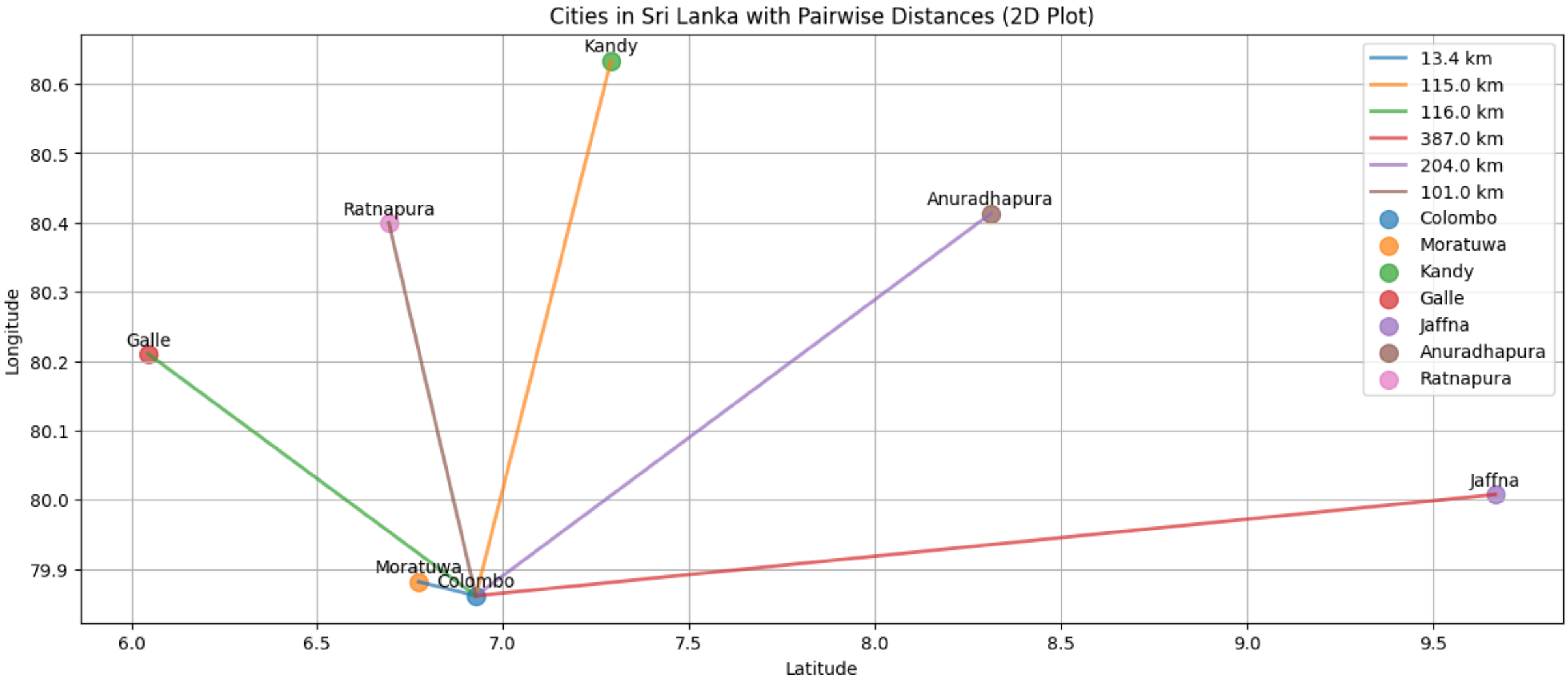
no defined continuous value

Multidimensional Scaling

City 1	City 2	Distance (km)
Colombo	Moratuwa	13.4
Colombo	Kandy	115.0
Colombo	Galle	116.0
Colombo	Jaffna	387.0
Colombo	Anuradhapura	204.0
Colombo	Ratnapura	101.0

Coordinates
Colombo: 6.927079, 79.861244
Moratuwa: 6.773, 79.8816
Kandy: 7.290572, 80.633726
Galle: 6.0461, 80.2103
Jaffna: 9.66845, 80.00742
Anuradhapura: 8.31223, 80.41306
Ratnapura: 6.6931, 80.3994

Multidimensional Scaling



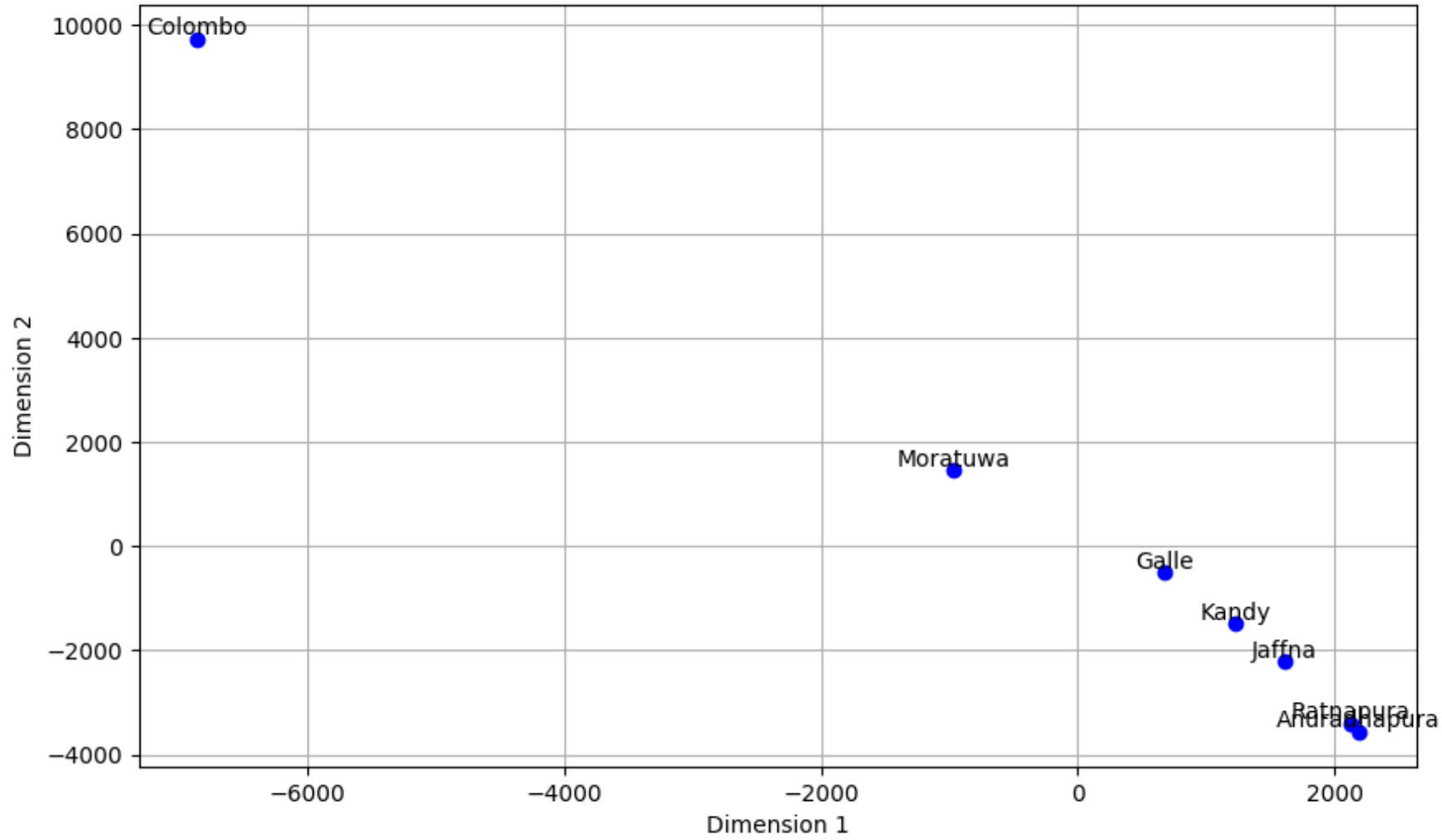
Multidimensional Scaling

City	Population (conurbation)	Population (municipality)	Area (km^2)	Density ($/km^2$)	Latitude	Longitude
Colombo	752,993	686,779	37.31	18,407	6.927079	79.861244
Moratuwa	207,755	189,750	23	8,250	6.773	79.8816
Kandy	125,351	123,952	27	4,591	7.290572	80.633726
Galle	99,478	97,101	7	5,712	6.0461	80.2103
Jaffna	88,138	76,080	20.2	3,766	9.66845	80.00742
Anuradhapura	63,208	83,312	36	2,314	8.31223	80.41306
Ratnapura	52,170	49,477	20	2,474	6.6931	80.3994

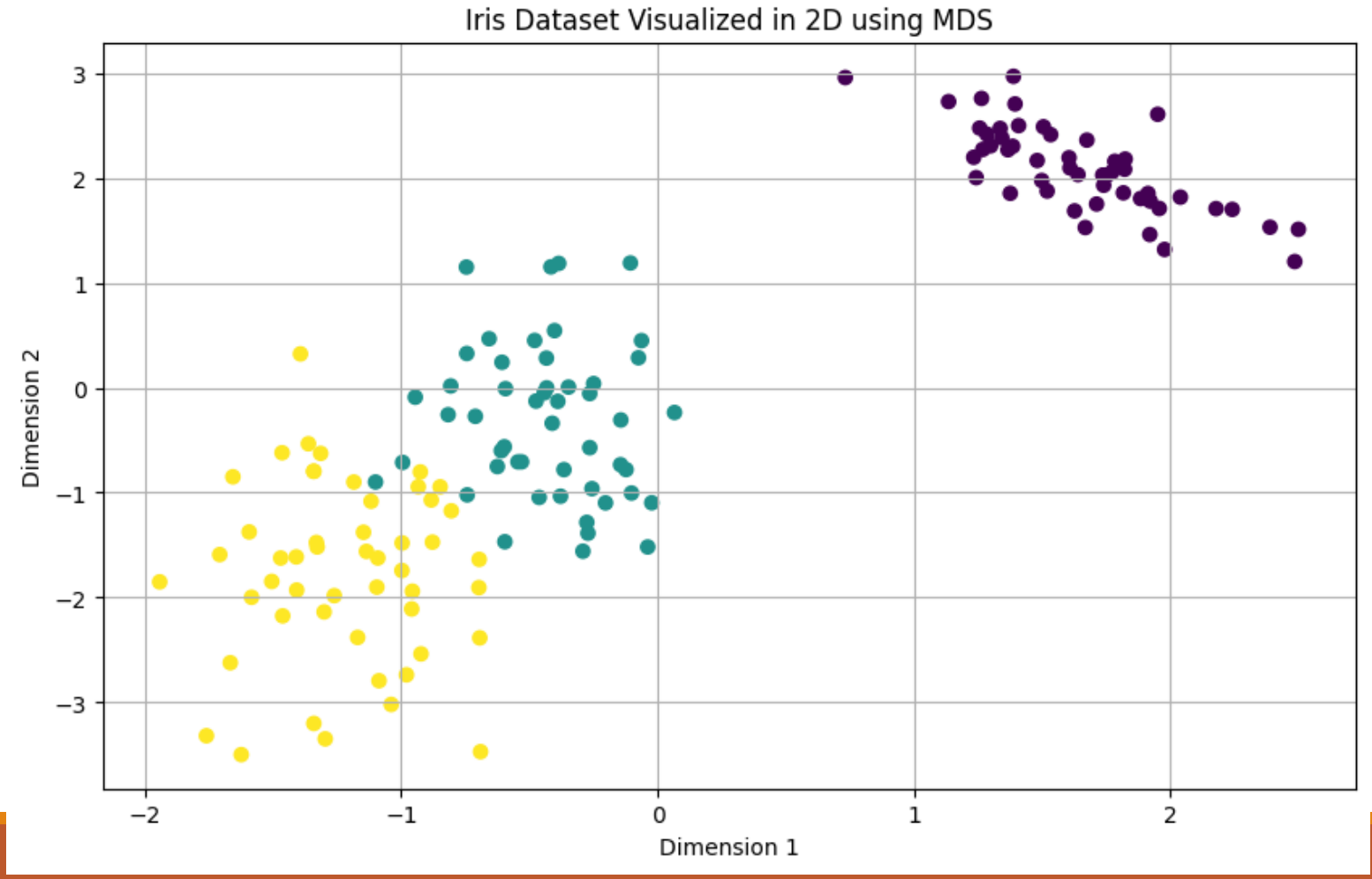
Now, we have more features.

How to visualize in 2d ? Multidimensional Scaling

Multidimensional Scaling



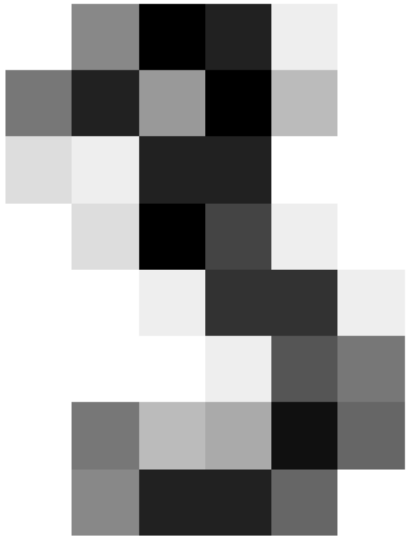
Multidimensional Scaling



Multidimensional Scaling

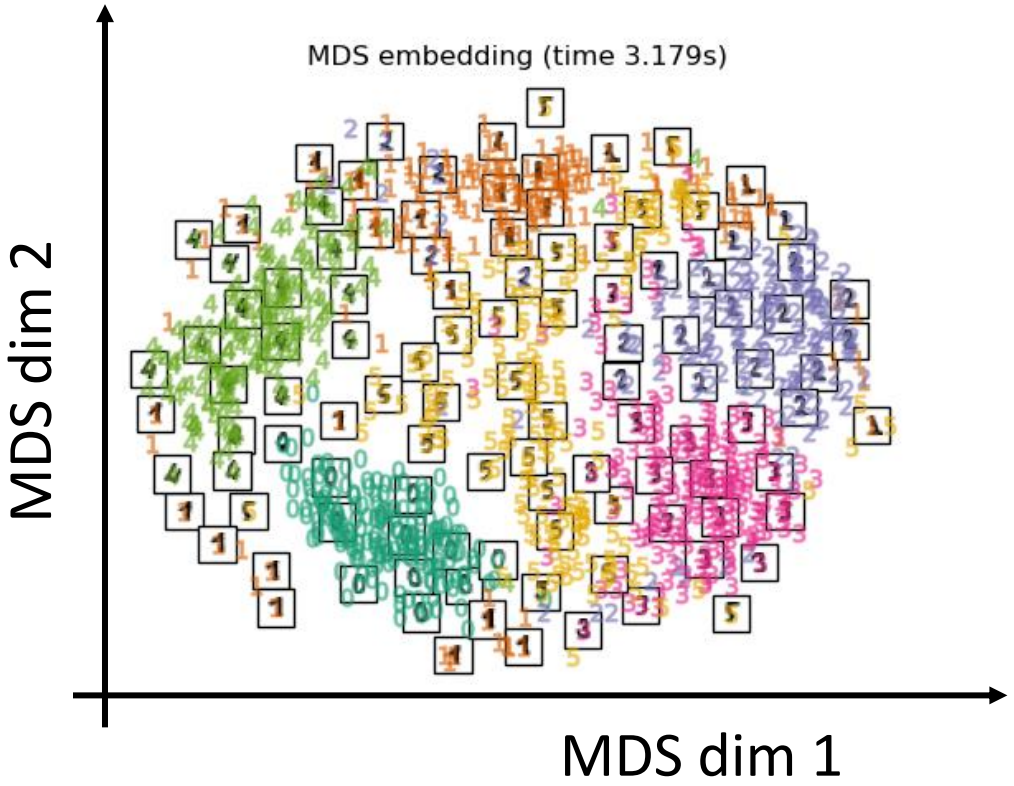


UCI digits dataset



(64-dimensional)

2d



Code: Manifold learning on handwritten digits: scikit-learn





Thank You

Q & A